

AppuntiEsame

167340

2025-01-21

MATRICI

Costruire una matrice A:

$$B = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 8 & 2 & 4 \end{bmatrix}$$

```
A=matrix(c(2,1,0,1,3,1,8,2,4),byrow=T,nrow=3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    2    1    0
## [2,]    1    3    1
## [3,]    8    2    4
```

```
# individuare l'elemento della prima riga e terza colonna nella matrice.
A[1,3]
```

```
## [1] 0
```

```
# seleziona tutti gli elementi della prima colonna della matrice.
A[,1]
```

```
## [1] 2 1 8
```

```
# seleziona tutti gli elementi della seconda riga della matrice.
A[2,]
```

```
## [1] 1 3 1
```

```
# selezionare tutti gli elementi della seconda e della terza riga della matrice.
A[2:3,]
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    1
## [2,]    8    2    4
```

```
# selezionare tutti gli elementi della prima e della seconda colonna della matrice.
A[,1:2]
```

```
##      [,1] [,2]
## [1,]    2    1
## [2,]    1    3
## [3,]    8    2
```

CREARE DATASET

```
# creazione dataset date liste di dati
nome=c("Rossi M","Bianchi G","Nicoletti C","Marcelli F","Petrone A")
età=c(32,39,46,28,51)
sesso=c("M","F","M","M","F")
titolo_di_studio=c("laurea","laurea","diploma","diploma","diploma")
attività=c("occupato","occupato","disoccupato","studente","casalinga")
peso=c(72,55,79,63,64)
data = data.frame(nome,età,sesso,titolo_di_studio,attività,peso)
head(data)
```

```
##      nome età sesso titolo_di_studio  attività peso
## 1  Rossi M  32    M      laurea    occupato   72
## 2 Bianchi G  39    F      laurea    occupato   55
## 3 Nicoletti C 46    M    diploma disoccupato   79
## 4 Marcelli F 28    M    diploma  studente   63
## 5 Petrone A 51    F    diploma  casalinga   64
```

```
# modalità della variabile sesso.
levels(factor(data$sesso))
```

```
## [1] "F" "M"
```

IMPORTARE DATASET

```
# importare su R il file "affitti.txt" .
affitti <- read.csv("affitti.txt", row.names=1, sep="")
# il file deve essere presente nella working directory
```

PRIME OPERAZIONI - AFFITTI

selezionare la variabile Capoluogo

```
affitti$Capoluogo
```

```
## [1] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A"
## [20] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "B" "B" "B" "B" "B" "B" "B" "B"
## [39] "B" "B" NA "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B"
## [58] "B" "B"
```

oppure

```
attach(affitti)
Capoluogo
```

```
## [1] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A"
## [20] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "B" "B" "B" "B" "B" "B" "B" "B"
## [39] "B" "B" NA "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B"
## [58] "B" "B"
```

```
# visualizzare le prime righe del dataset
head(affitti)
```

```
##      Affitto Capoluogo
## 1      170          A
## 2      200          A
## 3      230          A
## 4      240          A
## 5      250          A
## 6      270          A
```

```
# numero di osservazioni contenute nel dataset
nrow(affitti)
```

```
## [1] 59
```

```
# numero di variabili contenute nel dataset
ncol(affitti)
```

```
## [1] 2
```

```
# il nome delle variabili del dataset
names(affitti)
```

```
## [1] "Affitto" "Capoluogo"
```

```
# verificare se sono presenti dei dati mancanti
is.na(affitti$Affitto)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
is.na(affitti$Capoluogo)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
# eliminare i dati mancanti.
```

```
affitti2=na.omit(affitti)
```

```
affitti2
```

```
##      Affitto Capoluogo
## 1      170          A
## 2      200          A
## 3      230          A
## 4      240          A
## 5      250          A
## 6      270          A
## 7      290          A
## 8      325          A
## 9      360          A
## 10     390          A
## 11     330          A
## 12     370          A
## 13     400          A
## 14     420          A
## 15     380          A
## 16     390          A
## 17     350          A
## 18     360          A
## 19     390          A
## 20     400          A
## 21     450          A
## 22     440          A
## 23     430          A
## 24     370          A
## 25     360          A
## 26     350          A
## 27     340          A
## 28     400          A
## 29     290          A
## 30     380          A
## 31     200          B
## 32     230          B
## 33     250          B
## 34     240          B
## 35     300          B
## 36     290          B
## 37     320          B
## 38     330          B
## 39     350          B
## 40     360          B
```

```
## 42      390      B
## 43      300      B
## 44      330      B
## 45      340      B
## 46      350      B
## 47      380      B
## 48      290      B
## 49      260      B
## 50      250      B
## 51      230      B
## 53      310      B
## 54      320      B
## 55      280      B
## 56      370      B
## 57      380      B
## 58      330      B
## 59      280      B
```

OPERAZIONI DATASET - APPENDICE

```
appendiceA <- read.csv("appendiceA.txt", sep="")
attach(appendiceA)
```

```
##Head
```

```
# visualizzazione head
head(appendiceA)
```

```
##   Paziente  Peso  Colesterolo  Pressione  Sesso
## 1         1   147          213        127      F
## 2         2   156          174        116      M
## 3         3   112          193        110      F
## 4         4   127          196        110      F
## 5         5   144          220        130      F
## 6         6   140          183         99      M
```

```
# visualizzazione delle prime 10 righe
appendiceA[1:10,]
```

```
##   Paziente  Peso  Colesterolo  Pressione  Sesso
## 1         1   147          213        127      F
## 2         2   156          174        116      M
## 3         3   112          193        110      F
## 4         4   127          196        110      F
## 5         5   144          220        130      F
## 6         6   140          183         99      M
## 7         7   119          194        112      F
## 8         8   139          200        102      F
## 9         9   161          192        121      M
## 10        10   146          200        125      F
```

```
# numero di righe  
nrow(appendiceA)
```

```
## [1] 312
```

```
# numero di colonne  
ncol(appendiceA)
```

```
## [1] 5
```

```
##Frequenze
```

```
# frequenze assolute  
freq_ass=table(Sesso)  
freq_ass
```

```
## Sesso  
##      F      M  
## 148 164
```

```
# frequenze relative  
freq_rel=freq_ass/sum(freq_ass)  
freq_rel
```

```
## Sesso  
##           F           M  
## 0.474359 0.525641
```

```
# frequenze percentuali  
freq_perc=freq_rel*100  
freq_perc
```

```
## Sesso  
##           F           M  
## 47.4359 52.5641
```

```
##Variabili statistiche
```

```
# media  
mean(Peso,na.rm=T)
```

```
## [1] 152.5673
```

```
# mediana  
median(Peso,na.rm=T)
```

```
## [1] 150
```

```
# skewness
library(moments)
skewness(Peso,na.rm = T)
```

```
## [1] 0.2110351
```

```
# curtosi
kurtosis(Peso,na.rm = T)
```

```
## [1] 2.291515
```

```
# varianza
n = length(Peso)
varianza= var(Peso,na.rm=T)*(n-1)/n
```

```
# standard deviation
deviazione_standard=sd(Peso)*sqrt((n-1)/n)
```

```
# campo di variazione
campo_di_variazione=max(Peso)-min(Peso)
campo_di_variazione
```

```
## [1] 163
```

```
# campo di variazione
diff(range(Peso))
```

```
## [1] 163
```

```
##Quantili
```

```
# quantili
quartili=quantile(Peso,probs=c(0.25,0.5,0.75))
```

```
# scarto interquartile
as.numeric(quartili[3]-quartili[1])
```

```
## [1] 52
```

```
IQR(Peso)
```

```
## [1] 52
```

```
quantile(Peso,probs=0.01)
```

```
##      1%
## 98.11
```

```
quantile(Peso,probs=0.99)
```

```
##      99%  
## 224.56
```

Tutto Insieme

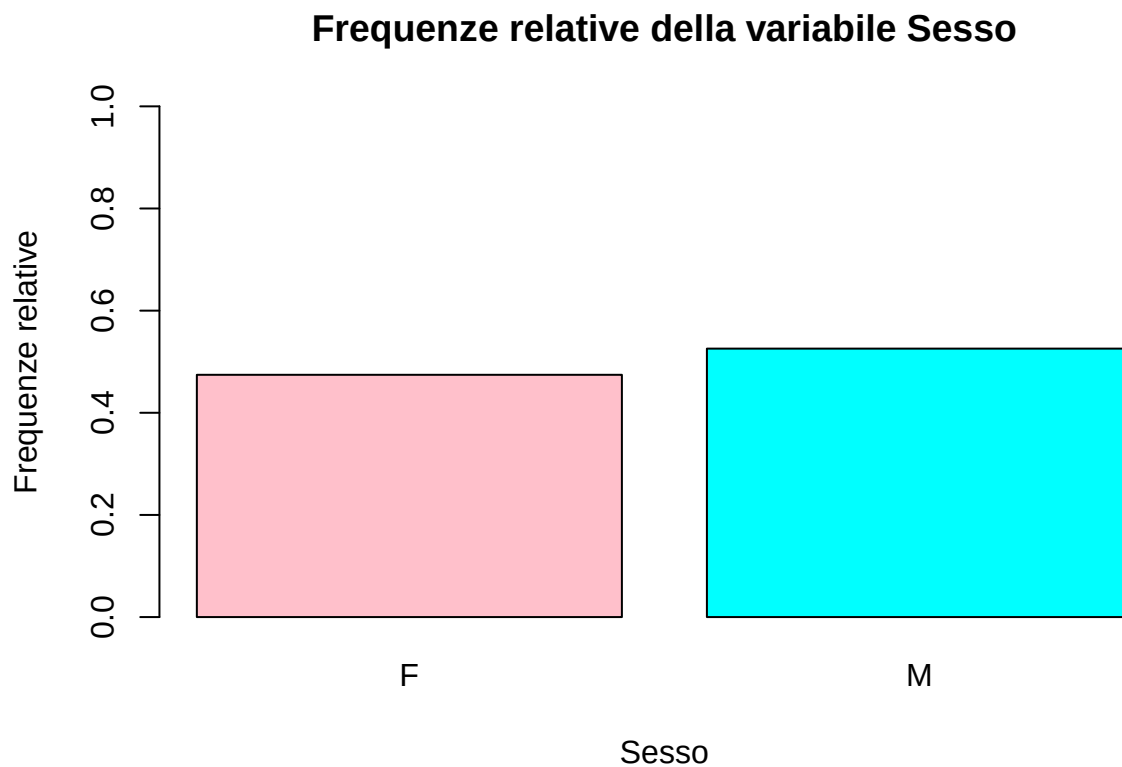
```
summary(Peso)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.  
##      86.0   126.0   150.0   152.6   178.0   249.0
```

```
##Diagrammi
```

```
# diagramma a barre
```

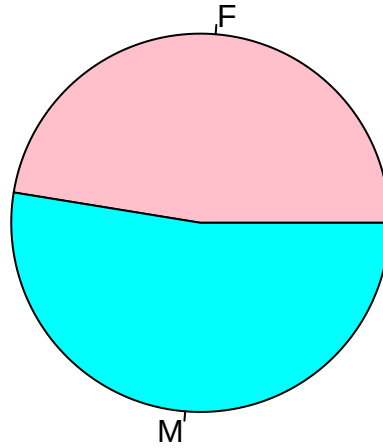
```
barplot(freq_rel,col=c("pink","cyan"),xlab="Sesso",ylab="Frequenze relative",main="Frequenze relative d
```



```
# diagramma a torta
```

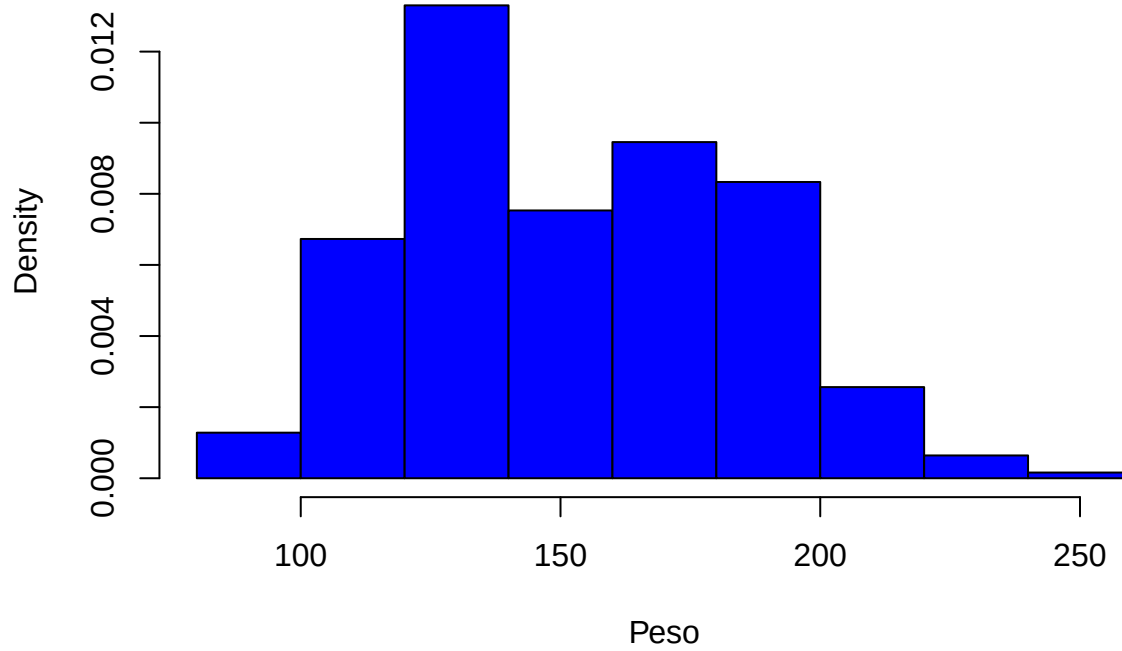
```
pie(freq_rel,col=c("pink","cyan"),main="Diagramma a torta per la variabile Sesso")
```

Diagramma a torta per la variabile Sesso



```
# istogramma  
hist(Peso, probability = T, main="Istogramma della variabile Peso", col="blue")
```

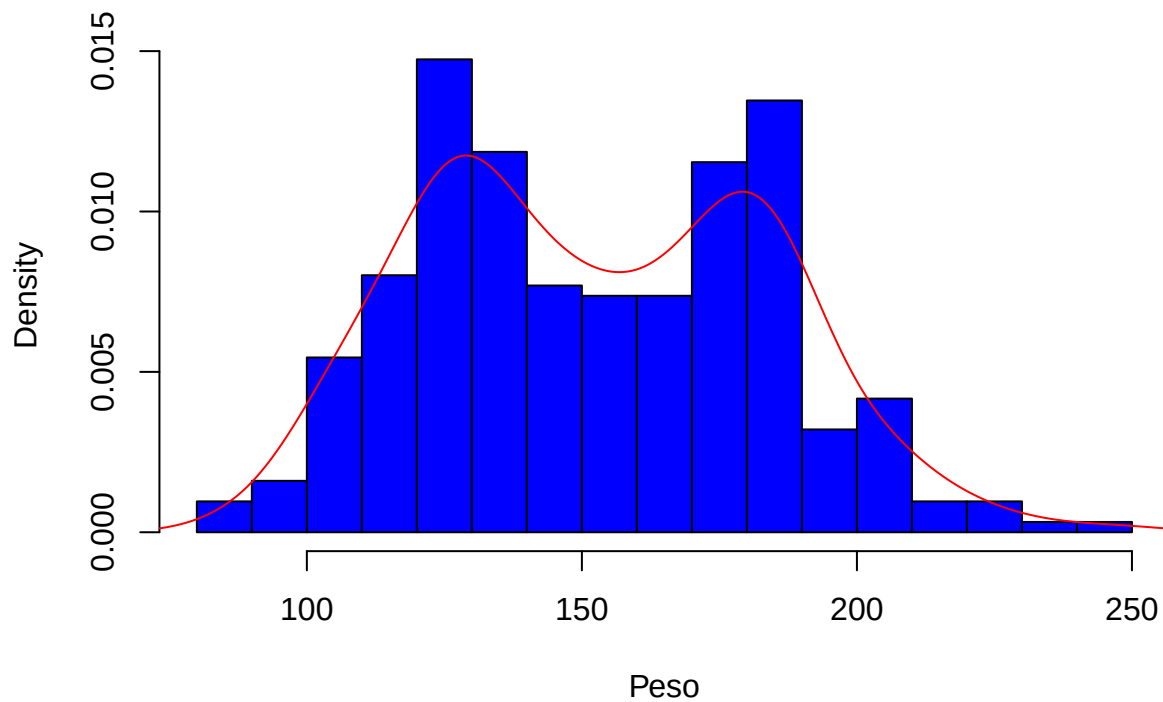
Istogramma della variabile Peso



```
#hist(Peso,freq=F)
```

```
# istogramma + linea di densità  
hist(Peso, probability = T, main="Istogramma della variabile Peso", col="blue",  
      breaks = seq(80, 250, 10))  
lines(density(Peso), col="red")
```

Istogramma della variabile Peso



```
# dati istogramma senza grafico
```

```
isto=hist(Peso, probability = T, plot = F)
```

```
## Warning in hist.default(Peso, probability = T, plot = F): argument
```

```
## 'probability' is not made use of
```

```
isto
```

```
## $breaks
```

```
## [1] 80 100 120 140 160 180 200 220 240 260
```

```
##
```

```
## $counts
```

```
## [1] 8 42 83 47 59 52 16 4 1
```

```
##
```

```
## $density
```

```
## [1] 0.0012820513 0.0067307692 0.0133012821 0.0075320513 0.0094551282
```

```
## [6] 0.0083333333 0.0025641026 0.0006410256 0.0001602564
```

```
##
```

```
## $mids
```

```
## [1] 90 110 130 150 170 190 210 230 250
```

```
##
```

```
## $xname
```

```
## [1] "Peso"
```

```
##
```

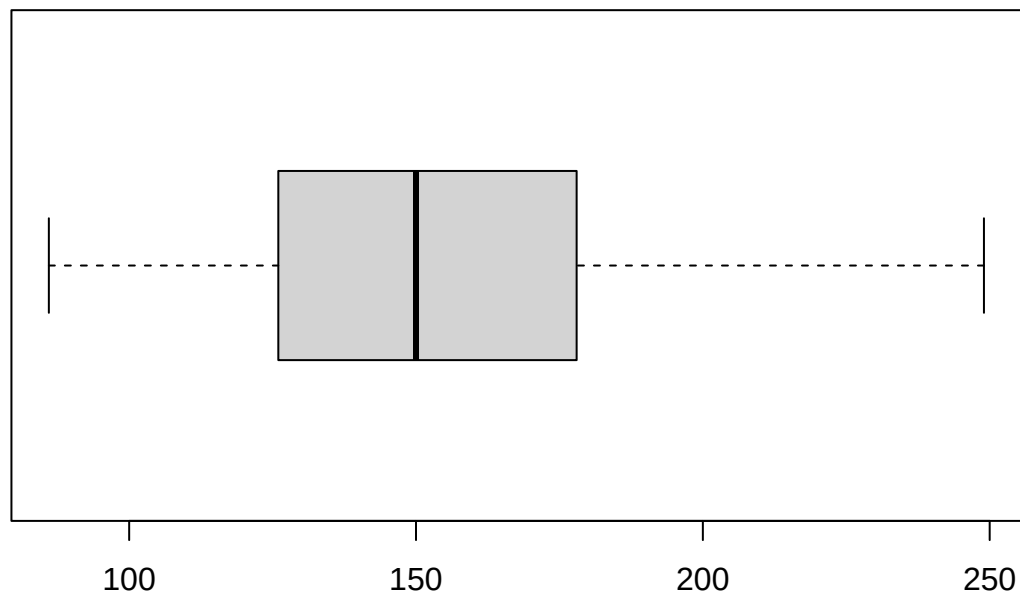
```
## $equidist
```

```
## [1] TRUE
##
## attr(,"class")
## [1] "histogram"
```

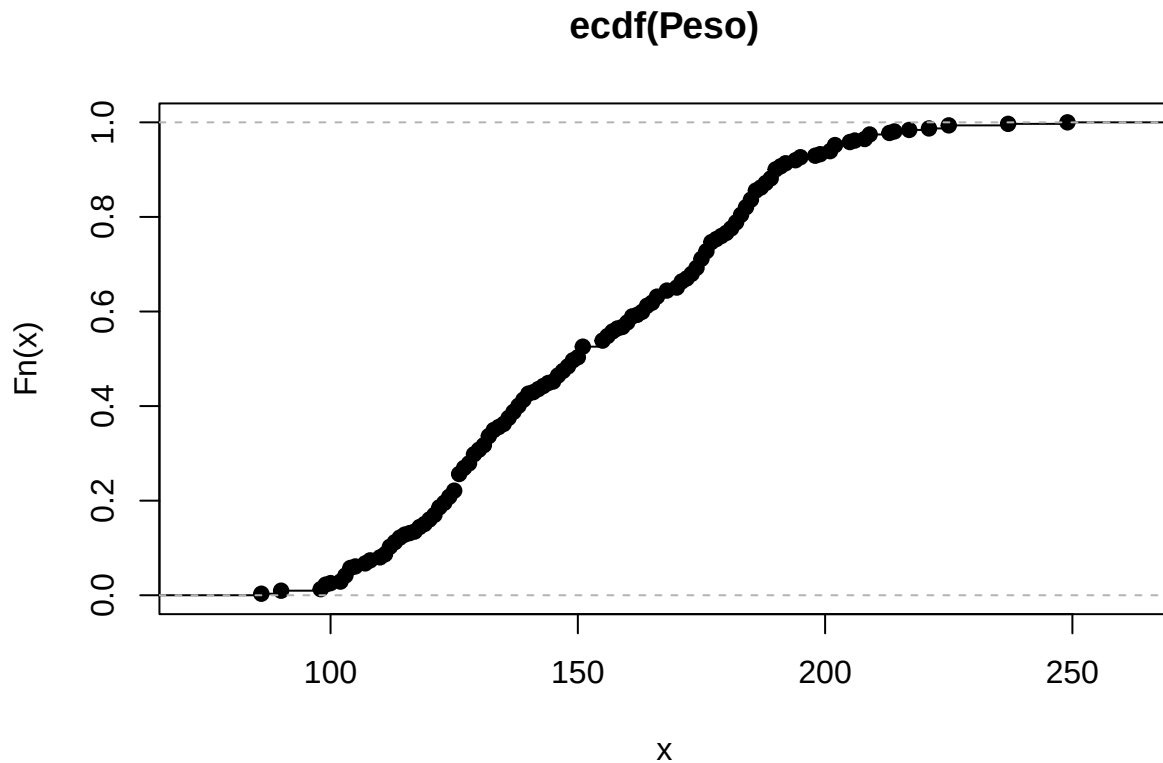
```
# densità per ogni colonna
isto$density
```

```
## [1] 0.0012820513 0.0067307692 0.0133012821 0.0075320513 0.0094551282
## [6] 0.0083333333 0.0025641026 0.0006410256 0.0001602564
```

```
# boxplot base (il più forte)
boxplot(Peso, horizontal = T)
```



```
# funzione di distribuzione empirica
plot(ecdf(Peso))
```



```
# campo di variazione con funzione
campodivariazione = function(x){
  camp = max(x)-min(x)
  return(camp)
}

campodivariazione(Peso)
```

```
## [1] 163
```

ESERCIZIO PATENTI

```
## Creazione dataset + frequenze
```

```
# creazione variabili
patente <- c('B', 'A', 'B', 'D', 'A1', 'B', 'C', 'B', 'B', 'B', 'B', 'A1', 'A', 'A', 'B', 'B', 'C', 'B',
esame <- c('P', 'B', 'P', 'P', 'B', 'B', 'P', 'P', 'P', 'B', 'P', 'P', 'B', 'B', 'B', 'P', 'P', 'P', 'P')

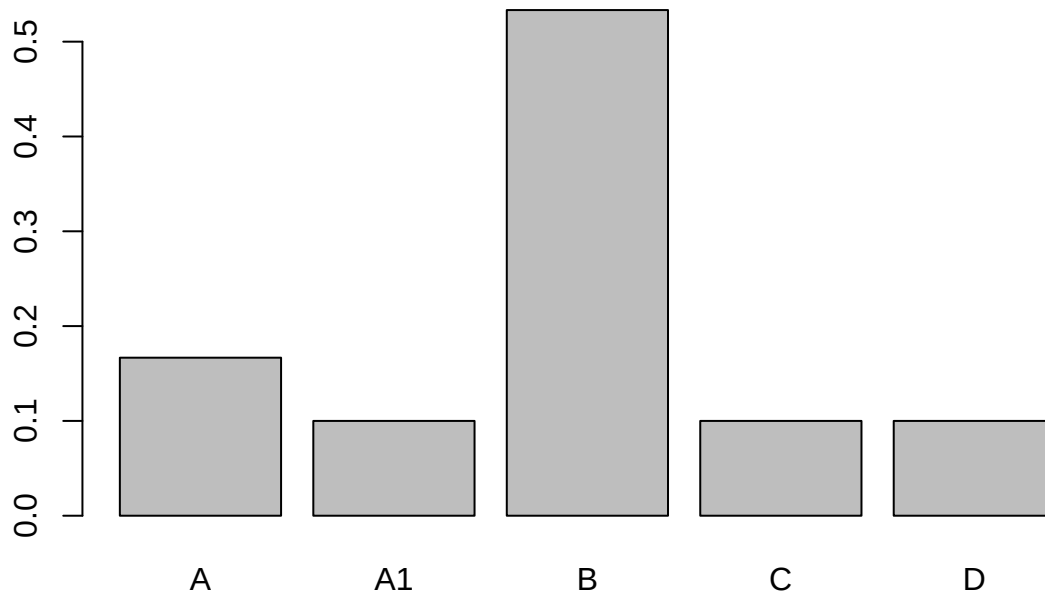
# frequenze assolute e relative delle due variabili
FA_patente=table(patente)
n = length(patente)
#n = sum(FA_patente)
FR_patente=FA_patente/n
FR_patente
```

```
## patente
##           A           A1           B           C           D
## 0.1666667 0.1000000 0.5333333 0.1000000 0.1000000
```

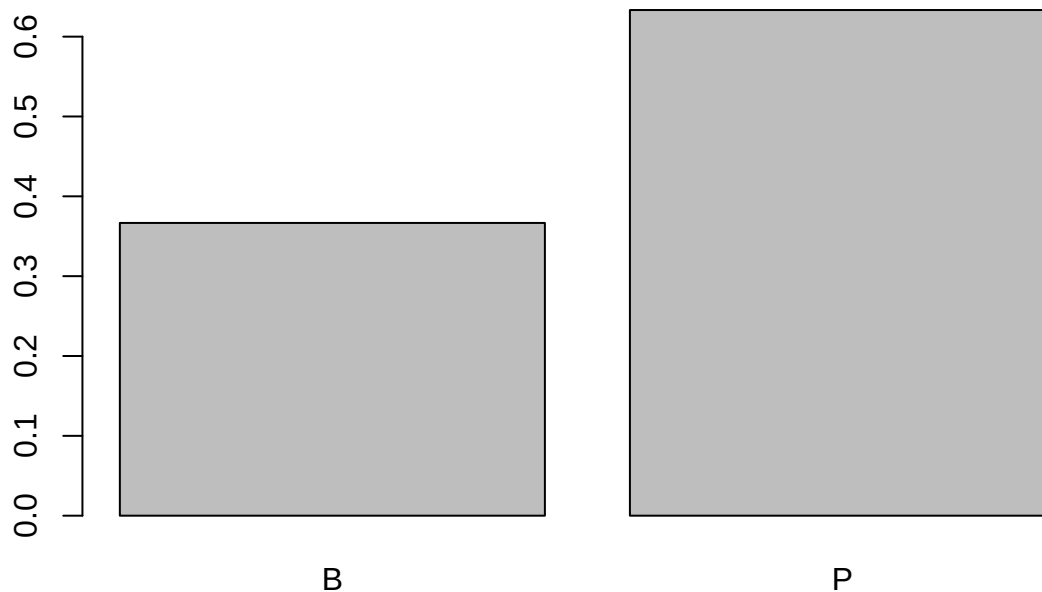
```
FA_esame=table(esame)
n = length(esame)
#n =sum(FA_esame)
FR_esame=FA_esame/n
FR_esame
```

```
## esame
##           B           P
## 0.3666667 0.6333333
```

```
# diagrammi a barre delle due variabili
barplot(FR_patente)
```



```
barplot(FR_esame)
```



```
# frequenze assolute e relative congiunte
```

```
FA_cong=table(esame,patente)
```

```
FA_cong
```

```
##      patente
## esame  A A1  B  C  D
##      B  5  2  3  0  1
##      P  0  1 13  3  2
```

```
FR_cong=FA_cong/n
```

```
FR_cong
```

```
##      patente
## esame      A      A1      B      C      D
##      B 0.1666667 0.0666667 0.1000000 0.0000000 0.0333333
##      P 0.0000000 0.0333333 0.4333333 0.1000000 0.0666667
```

[!] Freq marginali per riga (variabile esame)

```
apply(FA_cong,1,sum)
```

```
## B P  
## 11 19
```

```
# oppure  
margin.table(FA_cong,1)
```

```
## esame  
## B P  
## 11 19
```

[!] Freq marginali per colonna (variabile patente)

```
apply(FA_cong,2,sum)
```

```
## A A1 B C D  
## 5 3 16 3 3
```

```
#oppure  
margin.table(FA_cong,2)
```

```
## patente  
## A A1 B C D  
## 5 3 16 3 3
```

Indice chi-quadrato

```
chi_quadro = function(tab){  
  
  xtot = margin.table(tab,1) # marginali di riga  
  ytot = margin.table(tab,2) # marginali di colonna  
  
  n = sum(tab)  
  
  freq_attese = (xtot%*%t(ytot))/n  
  
  # in tab ci sono le frequenze osservate  
  chi_q= sum(((tab-freq_attese)^2)/freq_attese)  
  
  return(chi_q)  
}  
  
chi_quadro(FA_cong)
```

```
## [1] 13.76196
```

```
# oppure  
summary(FA_cong)
```

```
## Number of cases in table: 30  
## Number of factors: 2  
## Test for independence of all factors:  
##  Chisq = 13.762, df = 4, p-value = 0.008095  
##  Chi-squared approximation may be incorrect
```

```
# oppure  
chisq.test(FA_cong)
```

```
## Warning in chisq.test(FA_cong): Chi-squared approximation may be incorrect
```

```
##  
##  Pearson's Chi-squared test  
##  
## data:  FA_cong  
## X-squared = 13.762, df = 4, p-value = 0.008095
```

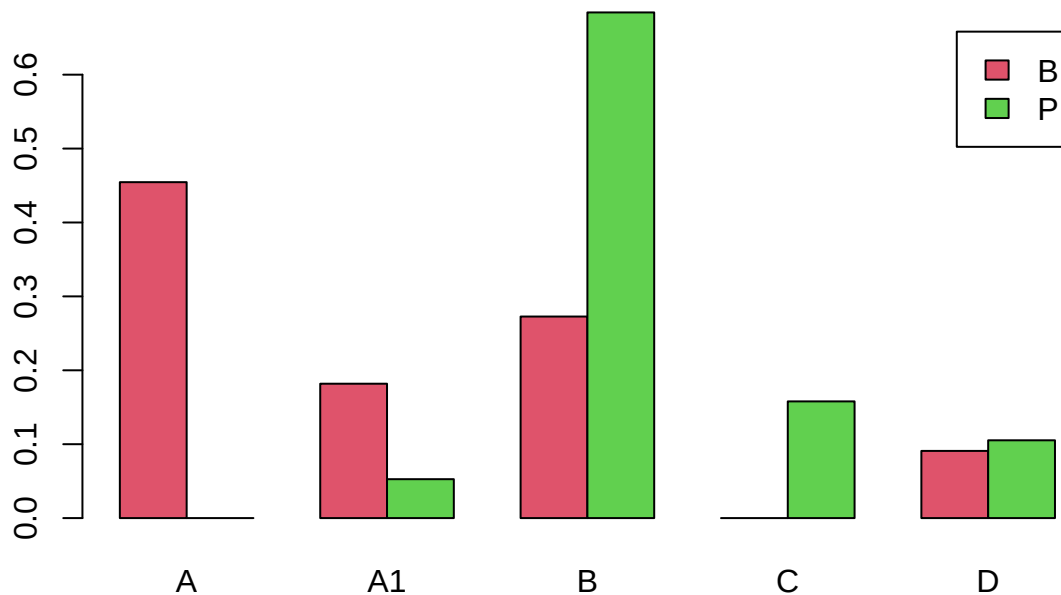
Chi-quadro normalizzato

```
# numero di colonne (numero modalità della variabile patente)  
s = length(levels(as.factor(patente)))  
# numero di righe (numero di modalità della variabile esame)  
r = length(levels(as.factor(esame)))  
chi_quadro(FA_cong)/(n*min((r-1),(s-1)))
```

```
## [1] 0.4587321
```

Diagramma a barra multiplo

```
freq.rel <- FA_cong/apply(FA_cong,1,sum)  
barplot(freq.rel,beside = T,col=c(2,3),legend.text = c("B","P"))
```



ESERCIZIO NEGOZI

```
# importare dataset
negozi <- read.csv("negozi.txt", row.names=1, sep="")
attach(negozi)
```

[!] Misure di posizione

```
# misure di posizione per la percentuale di femmine nei due Paesi

perc_femm_usa = perc_fem[stato=="USA"]
perc_femm_viet = perc_fem[stato=="VIETNAM"]

summary(perc_femm_usa)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  13.00   50.00   60.00   59.65   73.50   90.00
```

```
summary(perc_femm_viet)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      36.0   72.5   81.0   78.3   87.5   97.0
```

```
# oppure
tapply(perc_fem, stato, mean)
```

```
##      USA  VIETNAM
## 59.64706 78.29851
```

```
tapply(perc_fem, stato, summary)
```

```
## $USA
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      13.00   50.00   60.00   59.65   73.50   90.00
##
## $VIETNAM
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      36.0   72.5   81.0   78.3   87.5   97.0
```

[!] Misure di dispersione

```
# deviazione standard
sd(perc_femm_usa)
```

```
## [1] 18.38459
```

```
sd(perc_femm_viet)
```

```
## [1] 13.76206
```

```
# oppure
tapply(perc_fem, stato, sd)
```

```
##      USA  VIETNAM
## 18.38459 13.76206
```

```
# varianza
var(perc_femm_usa)
```

```
## [1] 337.993
```

```
var(perc_femm_viet)
```

```
## [1] 189.3944
```

```
# oppure
tapply(perc_fem, stato, var)
```

```
##      USA  VIETNAM
## 337.9930 189.3944
```

```
# scarto interquartile
IQR(perc_femm_usa)
```

```
## [1] 23.5
```

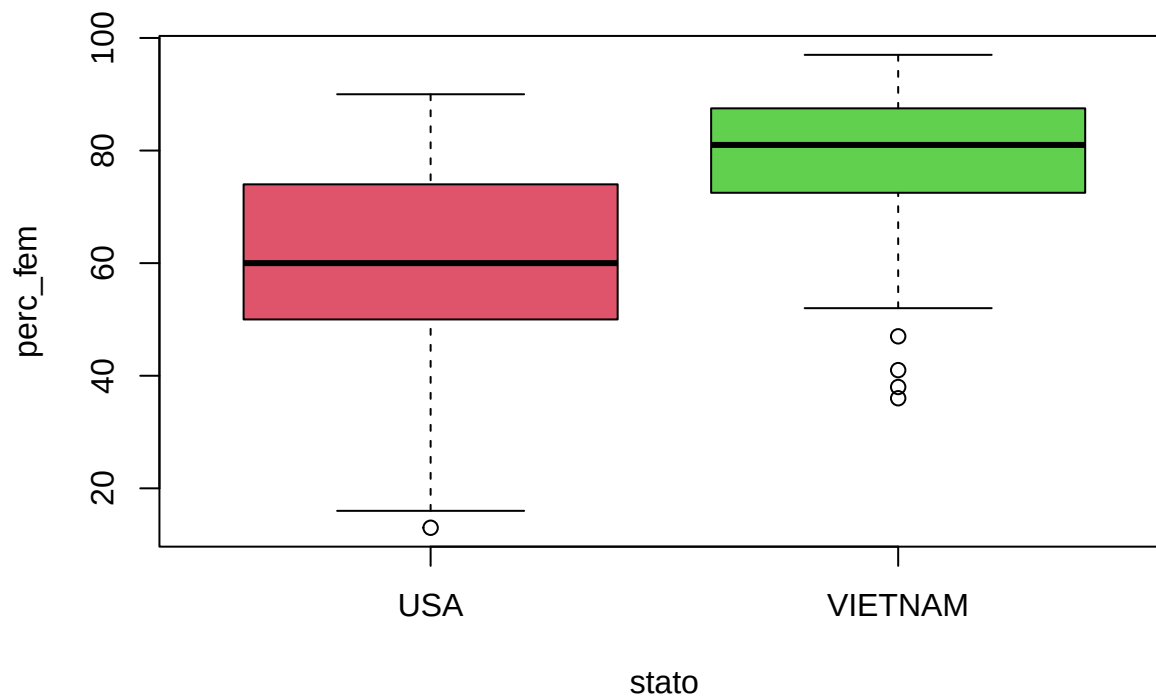
```
IQR(perc_femm_viet)
```

```
## [1] 15
```

```
# oppure
tapply(perc_fem, stato, IQR)
```

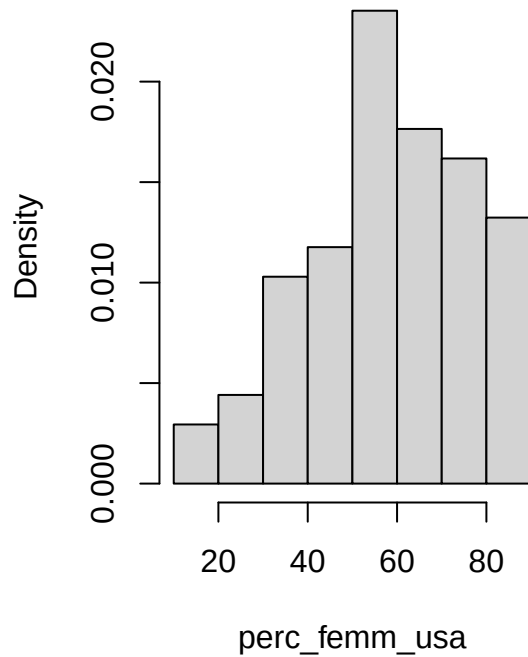
```
##      USA  VIETNAM
##   23.5   15.0
```

```
# boxplot
boxplot(perc_fem~stato,col=c(2,3))
```

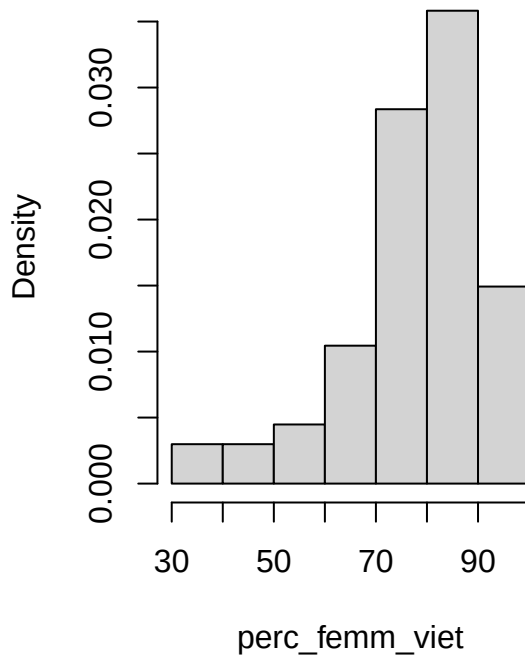


```
# istogramma
par(mfrow=c(1,2))
hist(perc_femm_usa,prob=T)
hist(perc_femm_viet,prob=T)
```

Histogram of perc_femm_usa



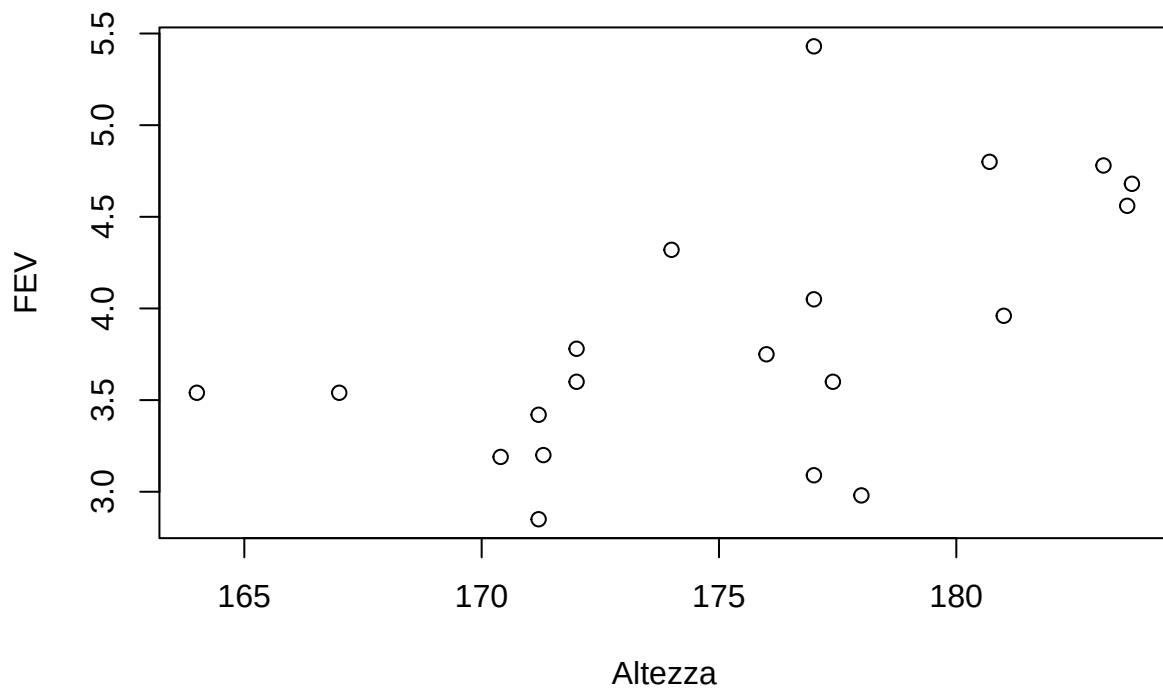
Histogram of perc_femm_viet



ESERCIZIO FEV

```
# importare dataset
fev <- read.csv("fev.txt", header=T, sep="")
attach(fev)
```

```
# diagramma di dispersione o scatterplot
plot(Altezza,FEV)
```



[!] Coefficienti di correlazione

```
# coefficiente di correlazione lineare di Pearson
cor(Altezza,FEV)
```

```
## [1] 0.5812976
```

```
# coefficiente di correlazione lineare di Spearman
cor(Altezza,FEV,method="spearman")
```

```
## [1] 0.5991711
```

```
# coefficiente di correlazione lineare di Kendall
cor(Altezza,FEV,method="kendall")
```

```
## [1] 0.412882
```

[!] Regressione

```
# covarianza fra due variabili  
cov(Altezza,FEV)
```

```
## [1] 2.256547
```

```
# b cappello  
(bhat=cov(Altezza,FEV)/var(Altezza))
```

```
## [1] 0.07438926
```

```
#a cappello  
(ahat=mean(FEV)-bhat*mean(Altezza))
```

```
## [1] -9.190389
```

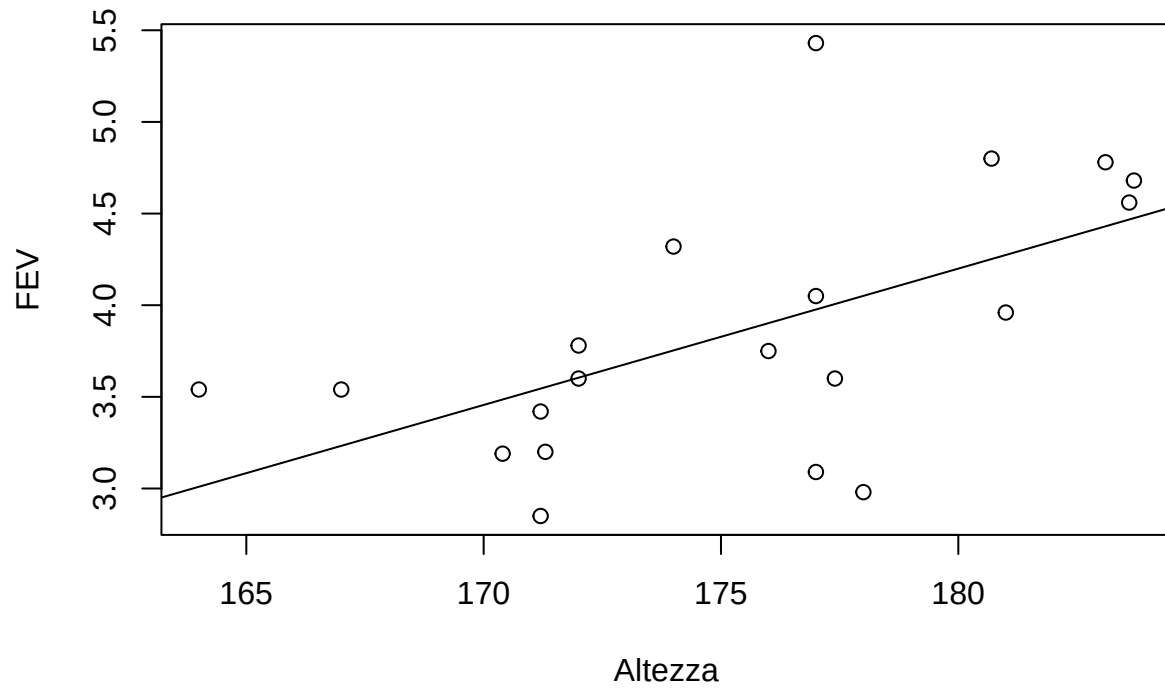
```
# modello lineare  
(mod=lm(FEV~Altezza,data=fev))
```

```
##  
## Call:  
## lm(formula = FEV ~ Altezza, data = fev)  
##  
## Coefficients:  
## (Intercept)      Altezza  
##      -9.19039      0.07439
```

```
# mod=lm(fev$FEV~fev$Altezza)  
  
summary(mod)
```

```
##  
## Call:  
## lm(formula = FEV ~ Altezza, data = fev)  
##  
## Residuals:  
##      Min       1Q   Median       3Q      Max   
## -1.07090 -0.32367  0.03446  0.31797  1.45349   
##  
## Coefficients:  
##              Estimate Std. Error t value Pr(>|t|)      
## (Intercept) -9.19039    4.30644  -2.134  0.04684 *      
## Altezza      0.07439    0.02454   3.031  0.00719 **     
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Residual standard error: 0.5892 on 18 degrees of freedom  
## Multiple R-squared:  0.3379, Adjusted R-squared:  0.3011   
## F-statistic: 9.187 on 1 and 18 DF,  p-value: 0.007185
```

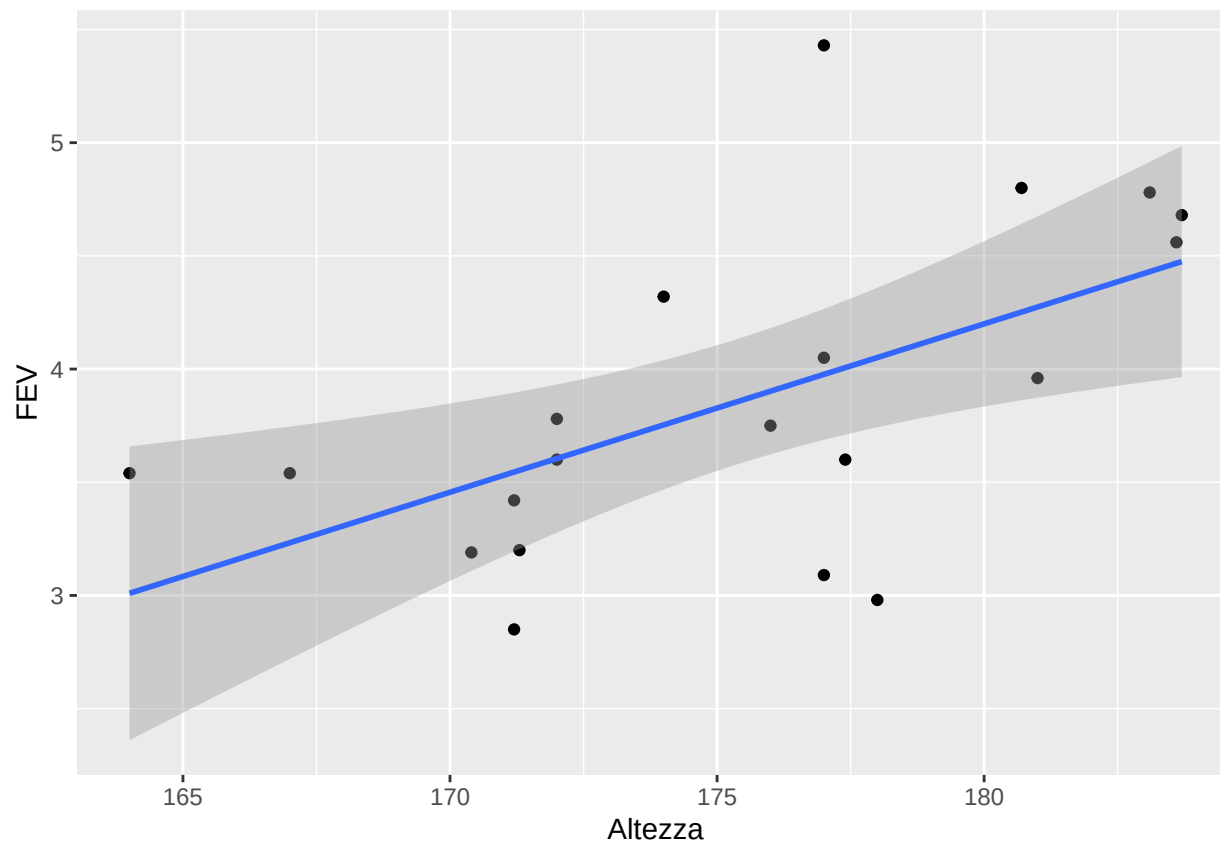
```
# plot della retta
plot(Altezza,FEV)
abline(mod)
```



```
# plot della retta con ggplot
library(ggplot2)

ggplot(fev,aes(x=Altezza,y=FEV))+
  geom_point()+
  geom_smooth(method="lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Coefficiente di correlazione lineare

```
# calcolo coefficiente
cor(Altezza,FEV)^2
```

```
## [1] 0.3379069
```

```
# estrazione di R^2 da summary
summary(mod)$r.squared
```

```
## [1] 0.3379069
```

```
# residui del modello
residuals(mod)
```

```
##          1          2          3          4          5          6
## 0.566657182 0.548249126 0.205081339 1.453489395 -0.886510605 0.175435706
##          7          8          9         10         11         12
## -0.152121343 0.073489395 0.530549803 -1.070899867 0.307382017 -0.125052884
##          13         14         15         16         17         18
## -0.406266309 -0.352491811 0.092520265 0.349714896 -0.004564294 -0.314067653
##          19         20
## -0.295541475 -0.695052884
```

Valori previsti

```
# calcolo dei valori previsti con a e b cappello
yhat=ahat+bhat*Altezza
yhat
```

```
## [1] 3.753343 4.251751 4.474919 3.976511 3.976511 3.604564 3.902121 3.976511
## [9] 3.009450 4.050900 3.232618 3.545053 4.006266 3.552492 4.467480 4.430285
## [17] 3.604564 4.274068 3.485541 3.545053
```

```
# calcolo dei valori previsti con a e b cappello
fitted.values(mod)
```

```
##      1      2      3      4      5      6      7      8
## 3.753343 4.251751 4.474919 3.976511 3.976511 3.604564 3.902121 3.976511
##      9     10     11     12     13     14     15     16
## 3.009450 4.050900 3.232618 3.545053 4.006266 3.552492 4.467480 4.430285
##     17     18     19     20
## 3.604564 4.274068 3.485541 3.545053
```

Coefficienti di regressione lineare

```
# calcolo coefficienti (intercept è FEV quando altezza = 0, Altezza = quanto varia
# FEV al variare)
coef(mod)
```

```
## (Intercept)      Altezza
## -9.19038879  0.07438926
```

[!] Previsione valore risposta dato valore indipendente x

```
# previsione con a e b cappello
ahat+bhat*184
```

```
## [1] 4.497235
```

```
# previsione con comando
predict(mod,newdata = data.frame(Altezza=184))
```

```
##      1
## 4.497235
```

```
# previsioni per 2 valori x e y
predict(mod,newdata = data.frame(Altezza=c(184,185)))
```

```
##      1      2
## 4.497235 4.571625
```

ESERCIZIO TEMPERATURA

```
# importare dataset
temperatura <- read.delim("temperatura.txt")
attach(temperatura)
```

```
## The following object is masked from appendiceA:
##
##      Sesso
```

```
table(Sesso)
```

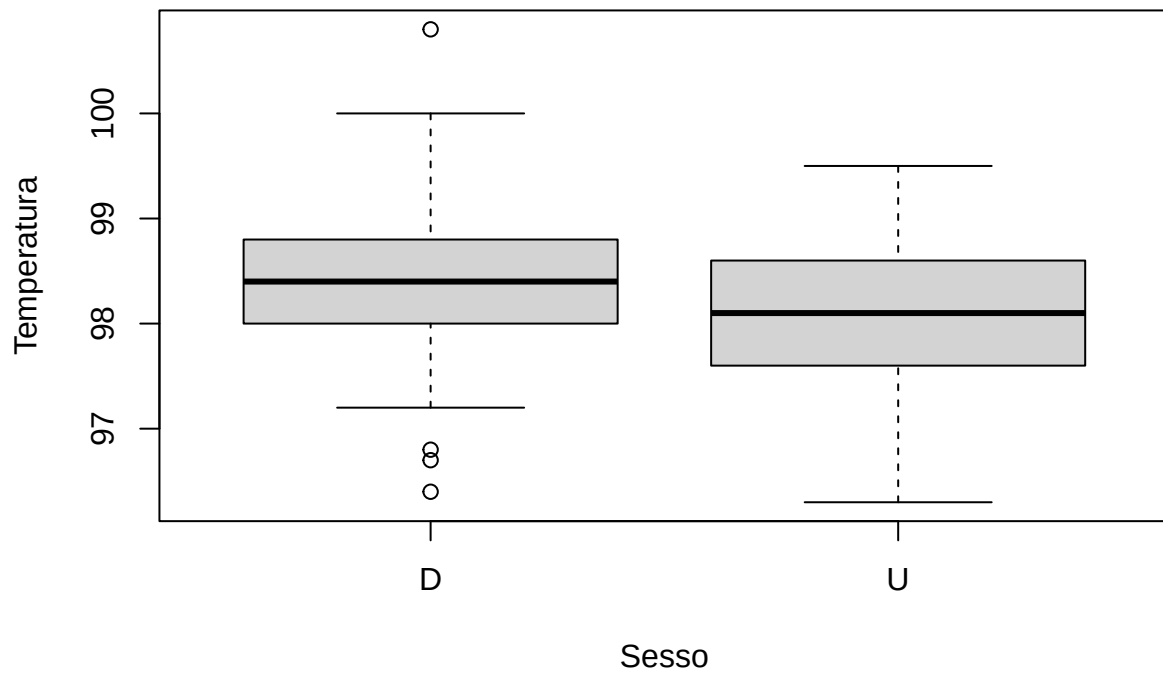
```
## Sesso
##  D  U
## 65 65
```

```
# summary per entrambi i sessi
tapply(Temperatura, Sesso, summary)
```

```
## $D
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   96.40   98.00   98.40   98.39   98.80  100.80
##
## $U
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   96.3    97.6    98.1    98.1    98.6    99.5
```

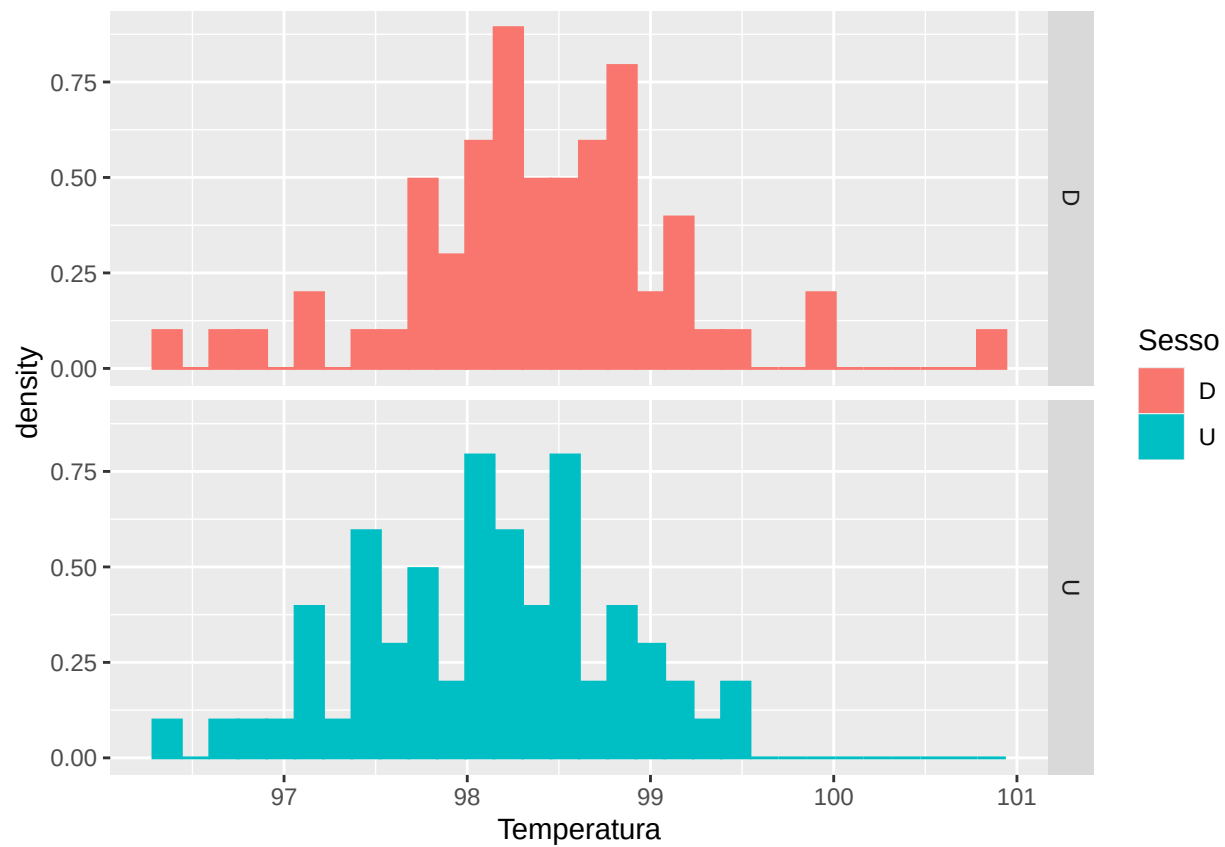
[!] Grafici

```
# boxplot
boxplot(Temperatura~Sesso)
```

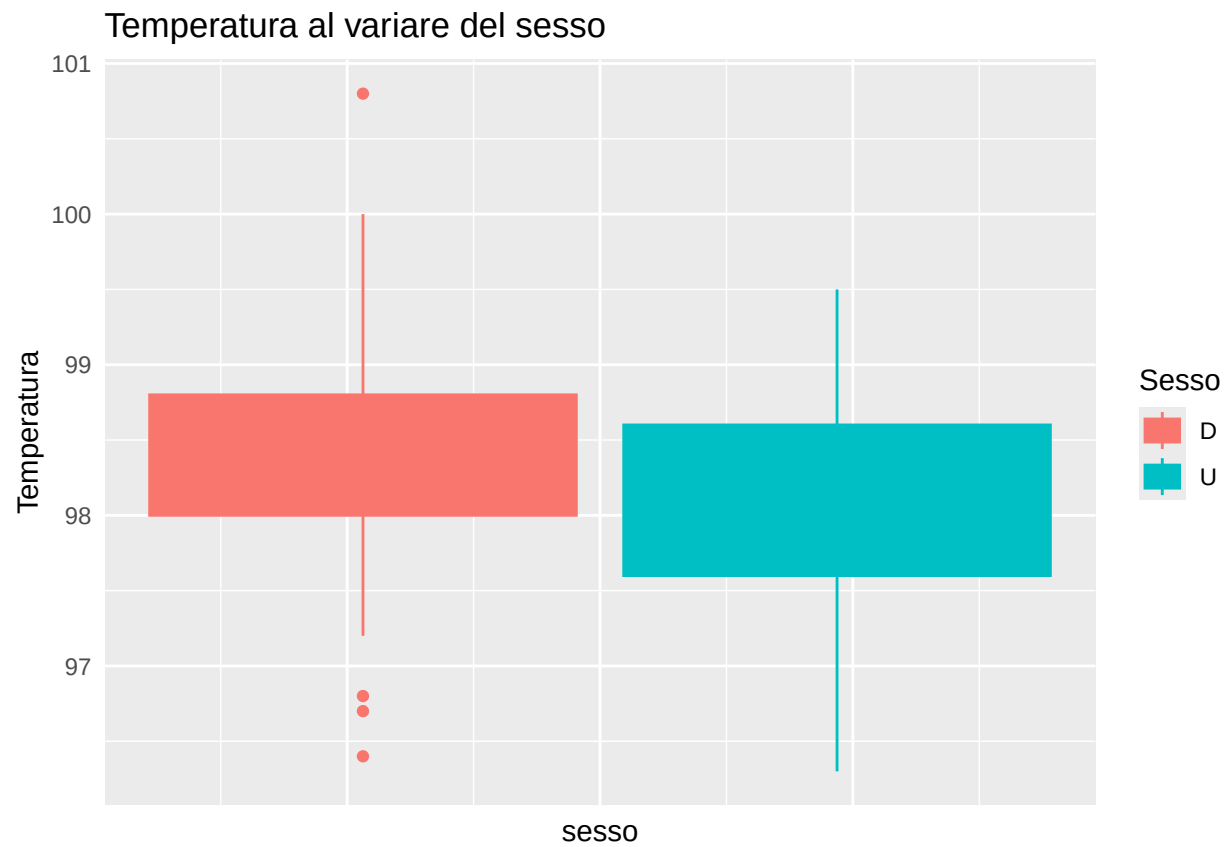


```
# Istogramma con ggplot
ggplot(temperatura, aes(x=Temperatura, y=after_stat(density), col=Sesso, fill=Sesso)) +
  geom_histogram() +
  facet_grid(Sesso ~ .)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

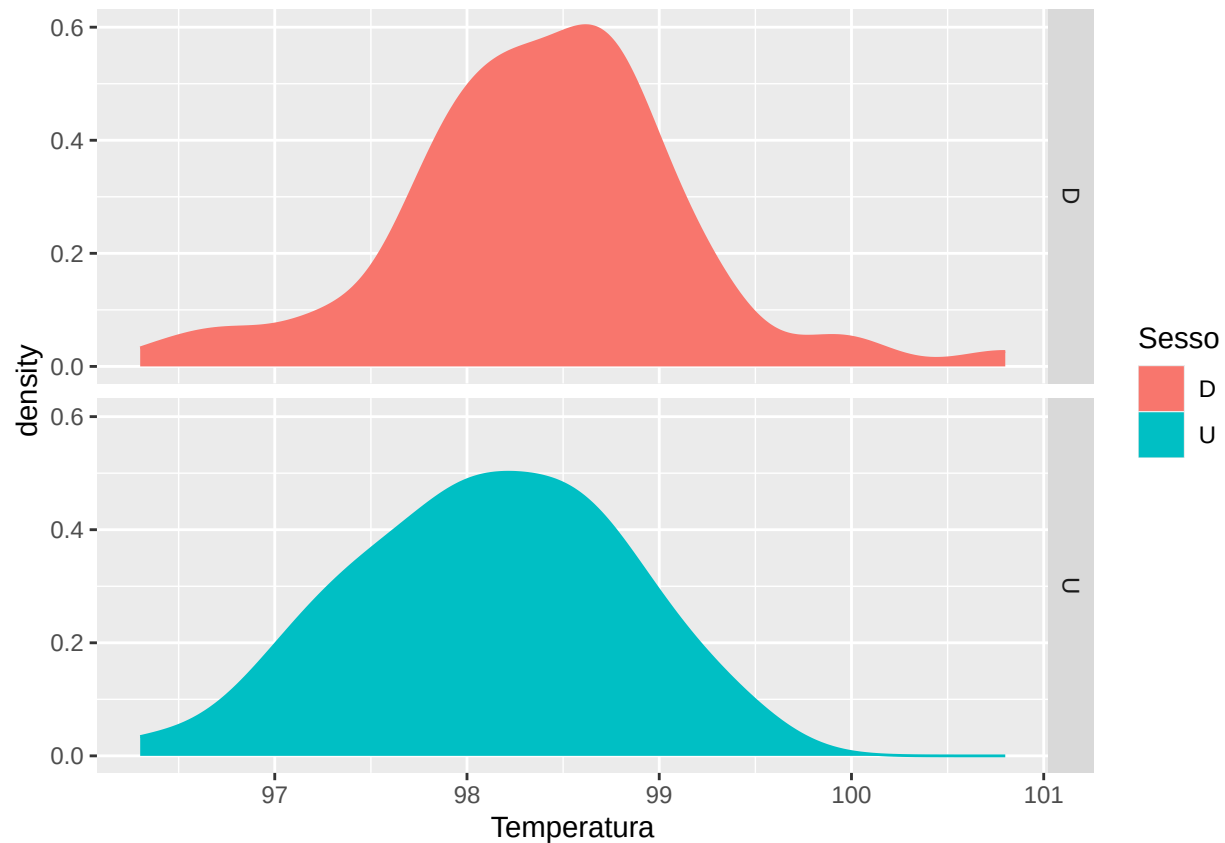


```
# Boxplot con ggplot
ggplot(temperatura,aes(y=Temperatura,col=Sesso,fill=Sesso))+geom_boxplot()+labs(title="Temperatura al v
```



Densità stimata con ggplot

```
ggplot(temperatura,aes(x=Temperatura,y=after_stat(density),col=Sesso,fill=Sesso))+geom_density()+facet_g
```



PROBABILITÀ - BASE

Probabilità semplice

```
#In una classe con 24 studenti, 7 studenti indossano jeans, 4 indossano pantaloncini, 8 indossano gonne
choose(7,2)*choose(5,1)/choose(24,3)
```

```
## [1] 0.05187747
```

```
#Una lotteria è costituita da 500 biglietti, di cui 6 vincenti. Si scelgono a caso 12 biglietti. Si vuole
choose(6,1)*choose(494,11)/choose(500,12)
```

```
## [1] 0.1287542
```

[!] Valori attesi

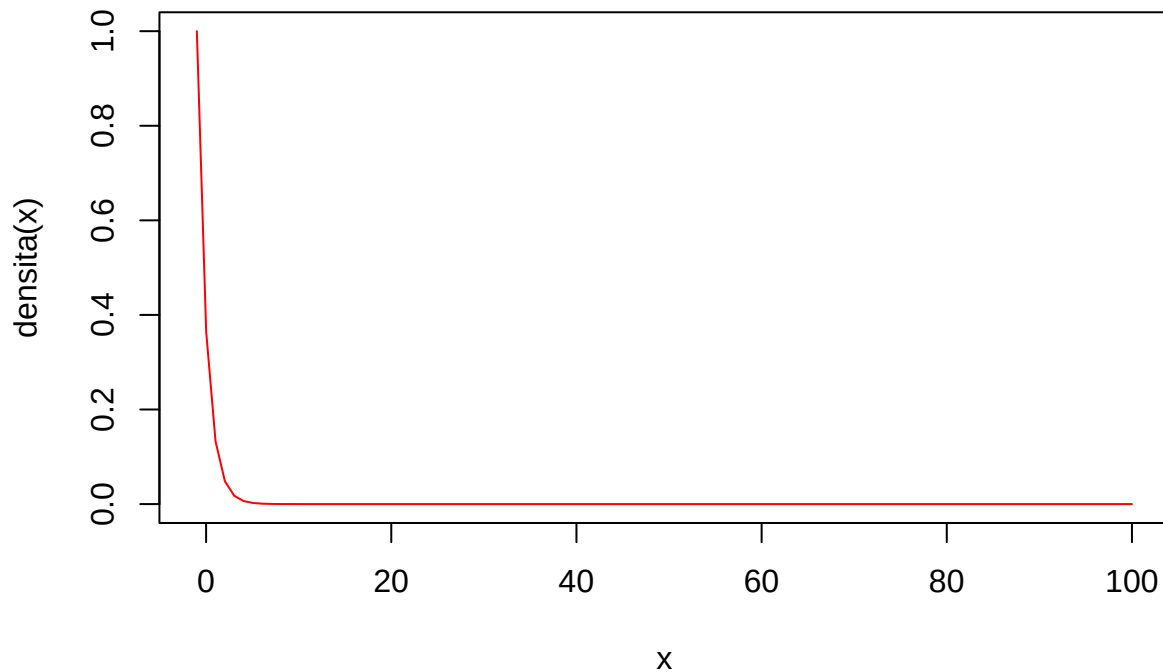
```
#Sia X una variabile casuale con supporto SX=[-1,+∞) e funzione di densità fX(x)=ke-x, se x ∈ SX, e nulla
to_integrate = function(x){
  exp(-x)
}
```

```
(k=1/integrate(to_integrate,-1,Inf)$value)
```

```
## [1] 0.3678794
```

```
densita = function(x){  
  k = 1/integrate(to_integrate,-1,Inf)$value  
  k*exp(-x)  
}
```

```
curve(densita(x),-1,100,col="red")
```



```
# valore atteso  
atteso = function(x){  
  x*densita(x)  
}  
  
(valore_atteso = integrate(atteso,-1,Inf)$value)
```

```
## [1] -8.314738e-13
```

```
## Moda  
(moda=optimise(densita,c(-1,100),maximum = T)$maximum)
```

```
## [1] -0.9999529
```

[!] Funzioni di ripartizione

```
# con calcolo teorico integrale
funz_rip = function(x){
  1-exp(-x-1)
}

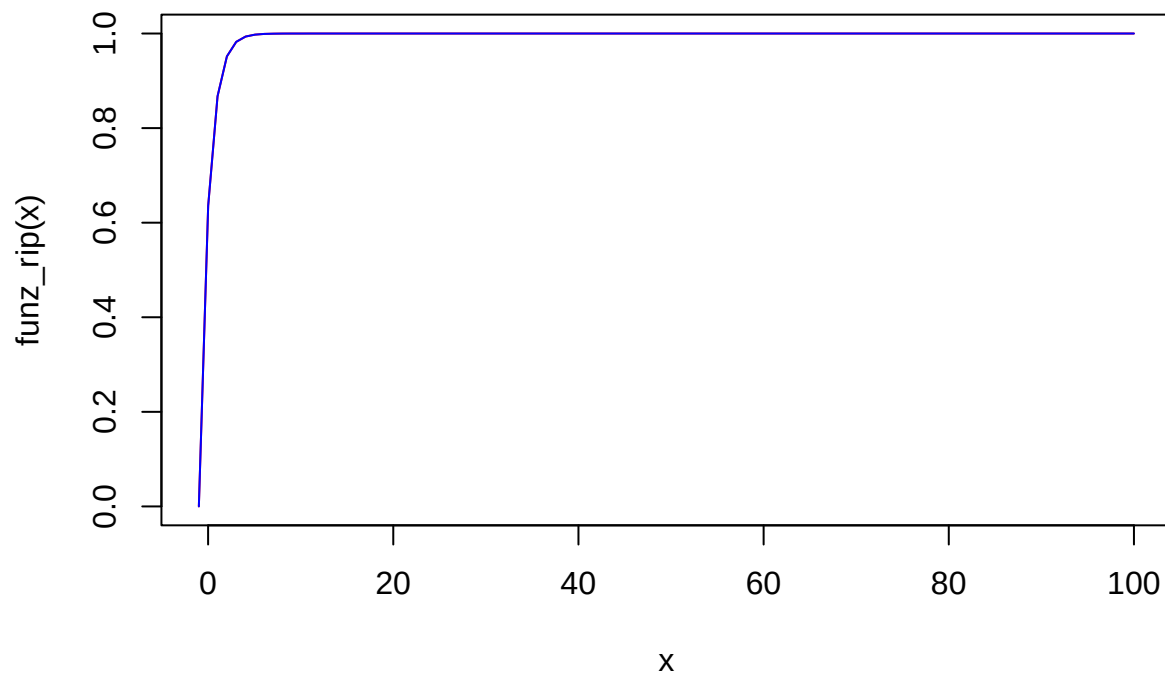
# calcolo con integrate
funz_rip2 = function(x){
  integrate(densita,-1,x)$value
}

funz_rip2_vect = Vectorize(funz_rip2,"x")
```

rappresentazione funzione di ripartizione

```
curve(funz_rip(x),-1,100,col="red")

curve(funz_rip2_vect(x),add=T,col="blue")
```



[!] Mediana

```
# con calcolo teorico  
(mediana=-1-log(0.5))
```

```
## [1] -0.3068528
```

```
## con la funzione di R uniroot  
to_solve = function(x){  
  funz_rip2(x)-0.5  
}  
  
(mediana=uniroot(to_solve,c(-1,100))$root)
```

```
## [1] -0.3068429
```

MODELLI PROBABILISTICI

Binomiale

```
# Un magazzino si rifornisce ogni settimana di un prodotto. La probabilità che nella settimana la doman  
  
n = 52  
p = 0.013  
  
# Qual è la probabilità che durante l'anno non venga mai superato lo stock?  
  
#  $P(X=0)$   
dbinom(0,size=n,prob=p)
```

```
## [1] 0.5063979
```

```
# Qual è la probabilità che nell'anno lo stock venga superato al massimo 2 volte?  
  
#  $P(X \leq 2) = P(X=0) + P(X=1) + P(X=2)$   
dbinom(0,n,p)+dbinom(1,n,p)+dbinom(2,n,p)
```

```
## [1] 0.9697216
```

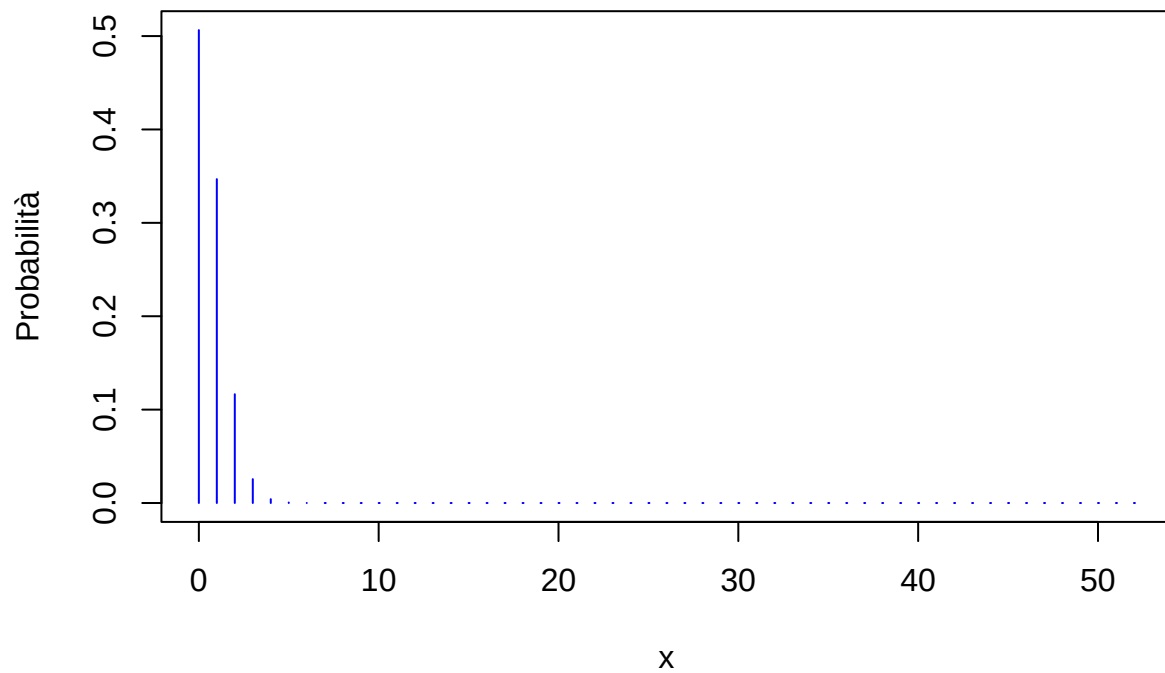
```
# oppure  
sum(dbinom(0:2,n,p))
```

```
## [1] 0.9697216
```

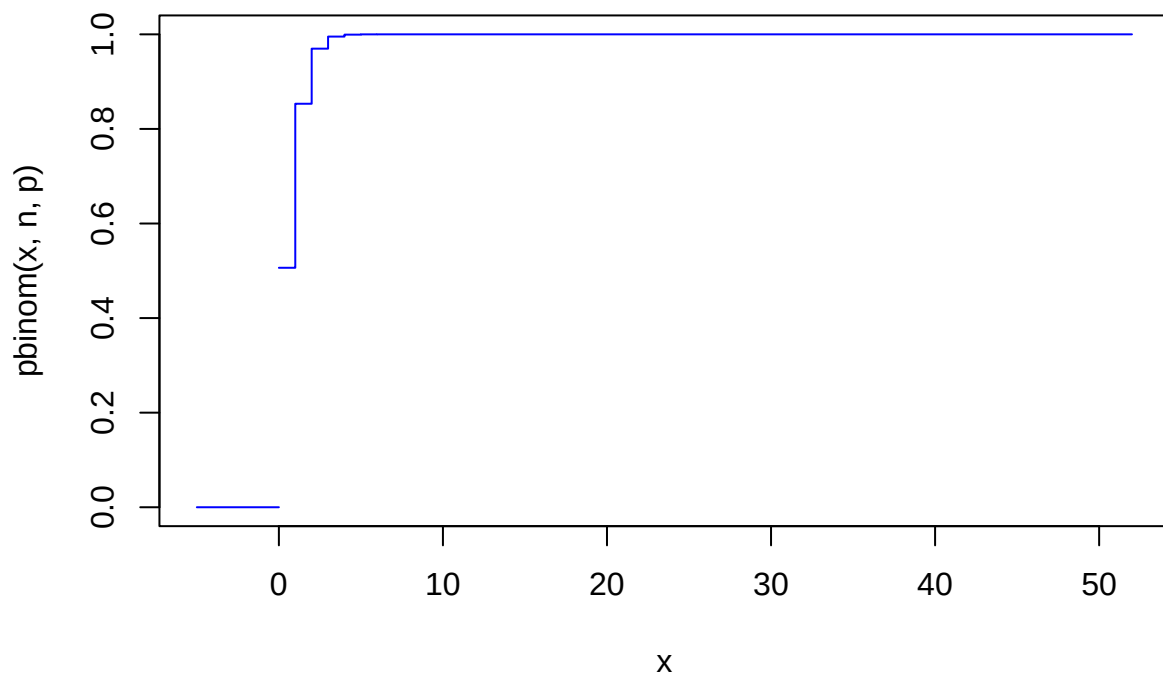
```
# oppure  
pbinom(2,n,p)
```

```
## [1] 0.9697216
```

```
# Rappresentazione funzione di probabilità
x = 0:52
plot(x,dbinom(x,n,p),type="h",col="blue",xlab="x",ylab="Probabilità")
```



```
# Rappresentazione funzione di ripartizione
x = 0:52
plot(x,pbinom(x,n,p),type="s",col="blue",xlab="x",ylim=c(0,1),xlim=c(-5,52))
segments(-5,0,0,0,col="blue")
```



Poisson

```
# Il periodo di vita in giorni delle particelle emesse da una sostanza radioattiva è una variabile di P

# X~Pois(lambda) vogliamo calcolare P(X<=2)
lambda=3.8
ppois(2,lambda)
```

```
## [1] 0.2688967
```

```
# oppure
sum(dpois(0:2,lambda))
```

```
## [1] 0.2688967
```

Geometrico

```
# Una macchina produce uno speciale transistor, componente necessario nella costruzione dei computer. L

p = 0.02# probabilità di successo, prob di essere difettoso
x = 9 # numero di fallimenti prima di arrivare al successo
```

```
# Qual è la probabilità che il decimo transistor prodotto sia il primo a essere difettoso?  
#  $X \sim \text{Geom}(p)$   $X$  conta il numero di insuccessi necessari al primo successo  
p*(1-p)^x
```

```
## [1] 0.01667496
```

```
dgeom(x,p)
```

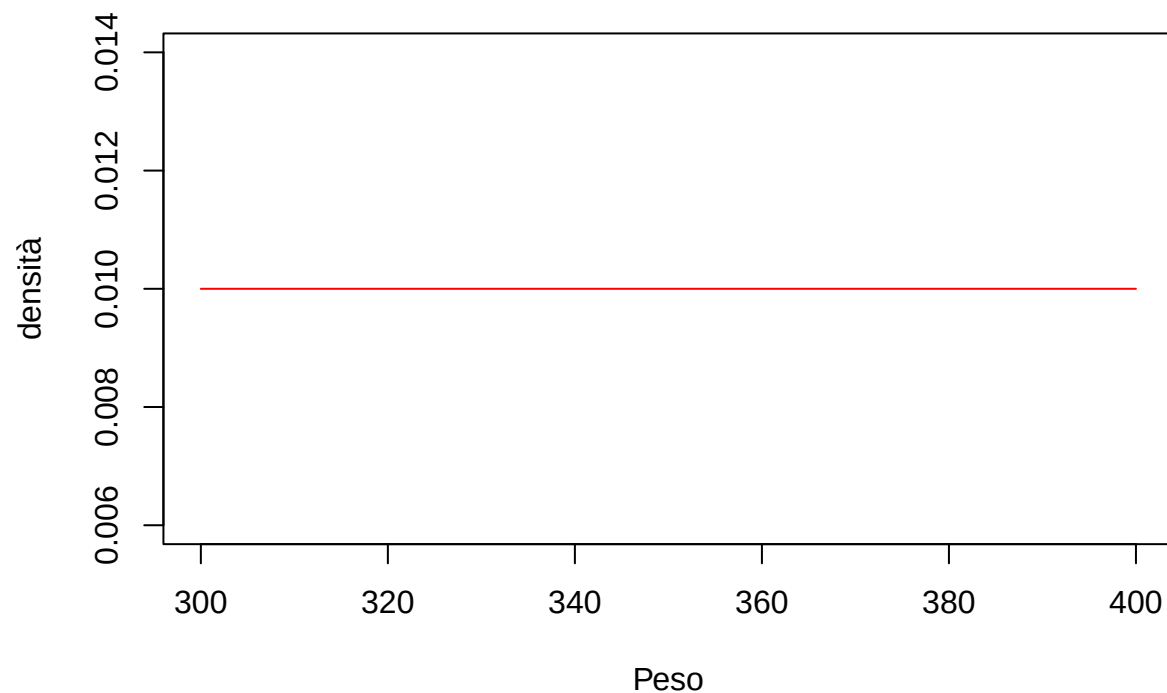
```
## [1] 0.01667496
```

Uniforme

```
# Da una ricerca riguardante gli allevamenti di bestiame si è osservato che la distribuzione del Peso d  
#  $X = \text{Peso dei bovini appartenenti all'intervallo } 300\text{--}400\text{Kg}$   
#  $X \sim U(300,400)$   
  
# Calcolare la probabilità di osservare un bovino con un peso tra 310kg e 350kg.  
  
#  $P(310 \leq X \leq 350)$   
punif(350,300,400)-punif(310,300,400)
```

```
## [1] 0.4
```

```
# Densità dell'uniforme tra 300 e 400  
curve(dunif(x,300,400),300,400,col="red",ylab="densità",xlab="Peso")
```



```
# Estrazione di 15 bovini con un peso compreso tra 300 e 400 Kg
set.seed(100)
runif(15,300,400)
```

```
## [1] 330.7766 325.7673 355.2322 305.6383 346.8549 348.3771 381.2403 337.0321
## [9] 354.6559 317.0262 362.4996 388.2166 328.0354 339.8488 376.2551
```

Esponenziale

```
# Il tempo che mediamente trascorre tra l'apertura di uno sportello pubblico e l'arrivo del primo utente

# X = tempo in minuti tra apertura sportello e arrivo primo utente
# lambda = 1/media
# X~Exp(lambda)

lambda = 1/4

# P(X>10)=1-P(X<=10)
1-pexp(10, rate = lambda)
```

```
## [1] 0.082085
```

Normale

```
# Il voto ottenuto al appello d'esame di Statistica segue una distribuzione normale con media =23 e var  
# X=Voto di statistica  $X \sim N(\mu=23, \sigma^2=9)$   
mu = 23  
sigma = sqrt(9)
```

```
# Determinare la probabilità di prendere un voto maggiore di 21 e inferiore a 25.
```

```
#  $P(21 \leq X \leq 25)$   
pnorm(25, mean = mu, sd = sigma) - pnorm(21, mean = mu, sd = sigma)
```

```
## [1] 0.4950149
```

```
# Visualizzare risultato  
library(tigerstats)
```

```
## Warning: package 'tigerstats' was built under R version 4.4.2
```

```
## Loading required package: abd
```

```
## Warning: package 'abd' was built under R version 4.4.2
```

```
## Loading required package: nlme
```

```
## Warning: package 'nlme' was built under R version 4.4.2
```

```
## Loading required package: lattice
```

```
## Loading required package: grid
```

```
## Loading required package: mosaic
```

```
## Warning: package 'mosaic' was built under R version 4.4.2
```

```
## Registered S3 method overwritten by 'mosaic':
```

```
##   method      from
```

```
## fortify.SpatialPolygonsDataFrame ggplot2
```

```
##
```

```
## The 'mosaic' package masks several functions from core packages in order to add
```

```
## additional features. The original behavior of these functions should not be affected by this.
```

```
##
```

```
## Attaching package: 'mosaic'
```

```
## The following objects are masked from 'package:dplyr':
##
##   count, do, tally

## The following object is masked from 'package:Matrix':
##
##   mean

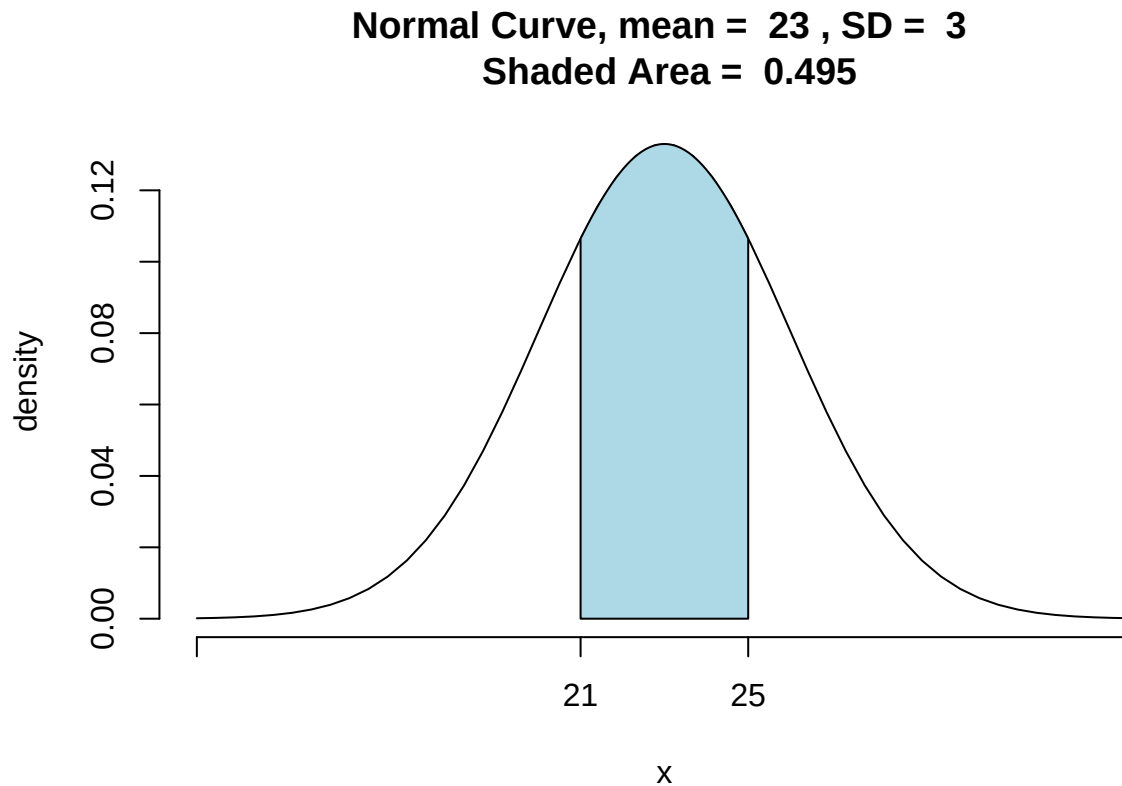
## The following object is masked from 'package:ggplot2':
##
##   stat

## The following objects are masked from 'package:stats':
##
##   binom.test, cor, cor.test, cov, fivenum, IQR, median, prop.test,
##   quantile, sd, t.test, var

## The following objects are masked from 'package:base':
##
##   max, mean, min, prod, range, sample, sum

## Welcome to tigerstats!
## To learn more about this package, consult its website:
## http://homerhanumat.github.io/tigerstats
```

```
pnormGC(c(21,25),mean=mu,sd=sigma,region="between",graph = T)
```



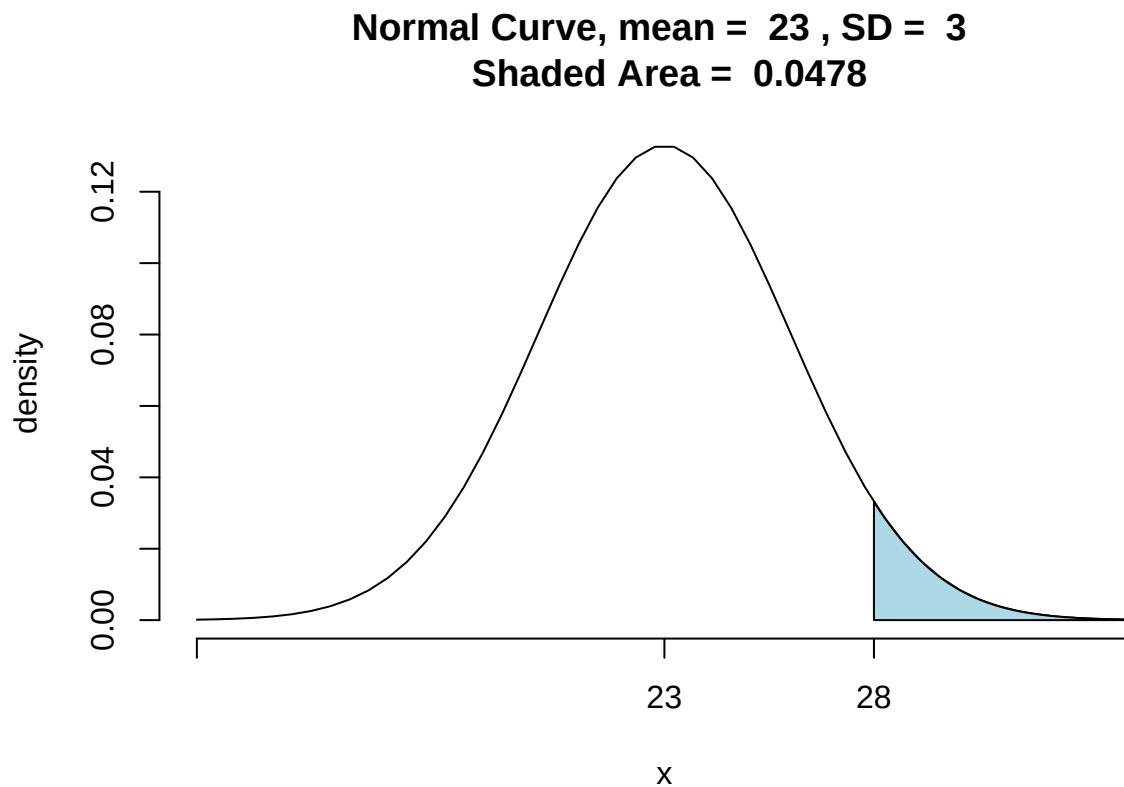
```
## [1] 0.4950149
```

```
# Determinare la probabilità di prendere un voto maggiore di 28.
```

```
#  $P(X > 28) = 1 - P(X \leq 28)$   
1-pnorm(28,mu,sigma)
```

```
## [1] 0.04779035
```

```
# oppure  
pnormGC(28,mu,sigma,graph=T,region="above")
```



```
## [1] 0.04779035
```

```
# Trovare il voto corrispondente al 95%.
```

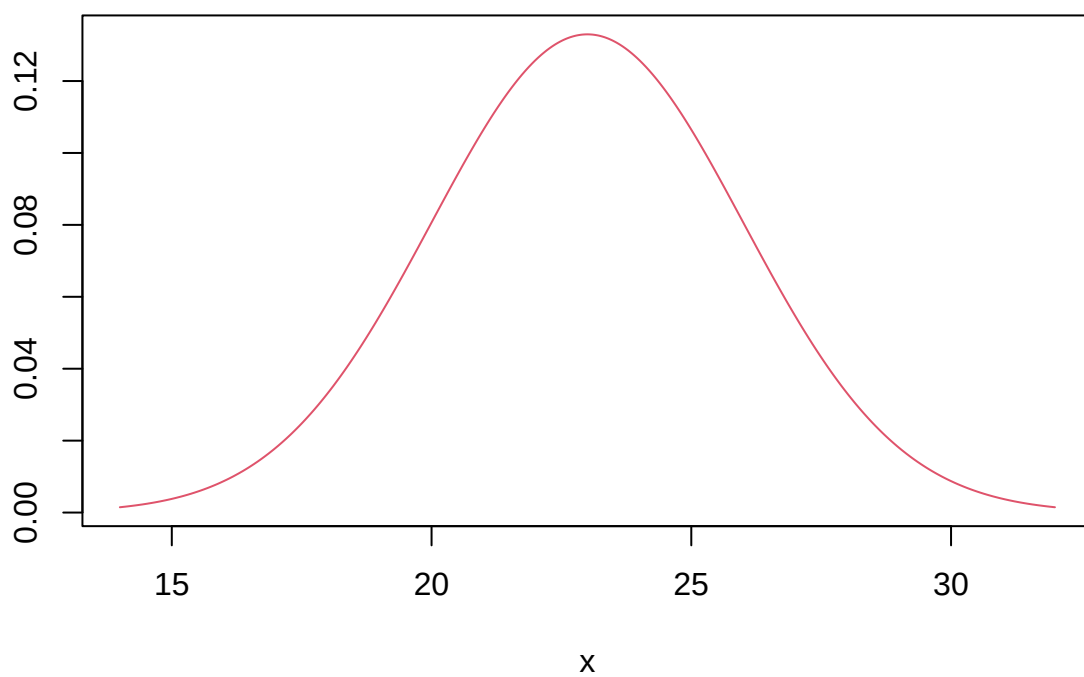
```
# quantile d'ordine 95%  
qnorm(0.95,mu,sigma)
```

```
## [1] 27.93456
```

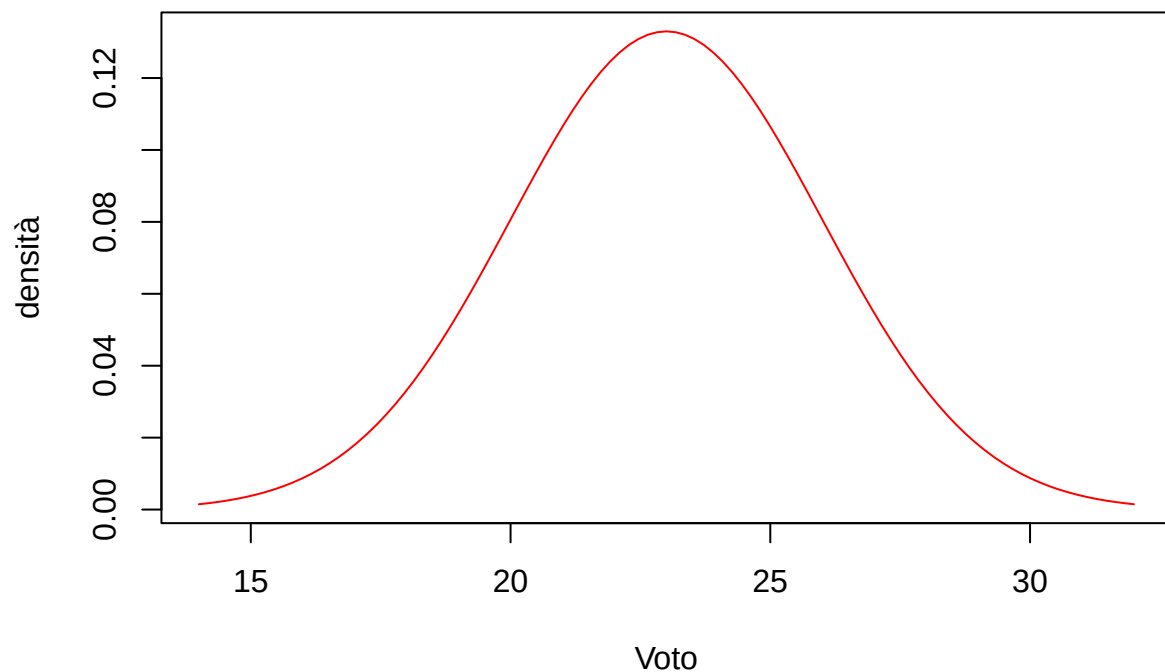
```
# Rappresentare la densità di probabilità.
```

```
x=seq(mu-3*sigma,mu+3*sigma,0.01)  
plot(x,dnorm(x,mu,sigma),col=2,main="Densità normale",ylab="",type = "l")
```

Densità normale



```
#oppure  
curve(dnorm(x,mu,sigma),mu-3*sigma,mu+3*sigma,col="red",ylab="densità",xlab="Voto")
```



STATISTICHE CAMPIONARIE

Campionaria

```
# Supponiamo che la percentuale di americani che sostengono l'espansione dell'energia solare sia p=0.88

N = 250000000
p = 0.88
n = 1000

# Crea un insieme di 250 milioni di risposte, di cui l'88% sono "supporto" e il restante sono "non supp
risposte = c(rep("Supporto",N*p), rep("Non Supporto",N*(1-p)))

# Estrai dalla popolazione 1000 risposte senza sostituzione.
campione = sample(risposte,1000)

# Calcolare la proporzione campionaria.
(phat = sum(campione=="Supporto")/1000)

## [1] 0.862
```

```
# oppure  
mean(campione=="Supporto")
```

```
## [1] 0.862
```

```
# oppure  
as.numeric(table(campione)[2]/1000)
```

```
## [1] 0.862
```

```
# Ripeti l'operazione 1000 volte e rappresenta tramite un grafico il risultato della simulazione, aggiu
```

```
n_camp = 1000
```

```
p_hat = NULL
```

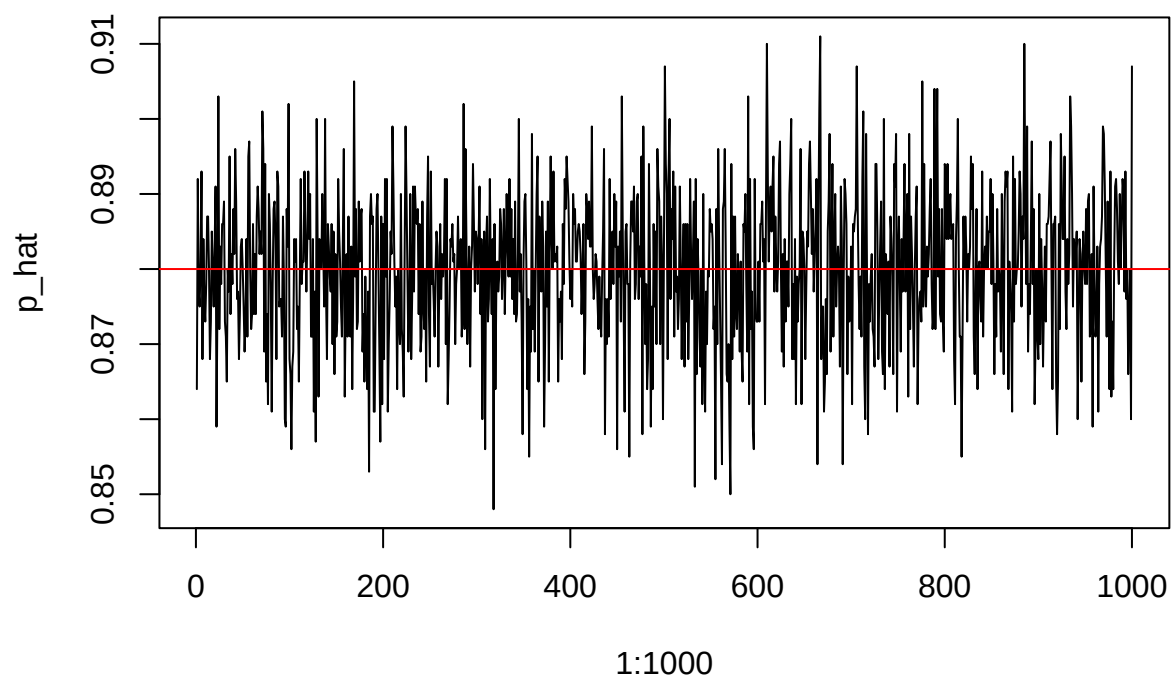
```
# Calcolo proporzione campionaria
```

```
for(i in 1:n_camp){  
  campione = sample(risposte,1000)  
  p_hat[i] = mean(campione=="Supporto")  
}
```

```
# Grafico di confronto
```

```
plot(1:1000,p_hat,type="l")
```

```
abline(h=p,col="red")
```



```
# media della proporzione campionaria  
mean(p_hat)
```

```
## [1] 0.879977
```

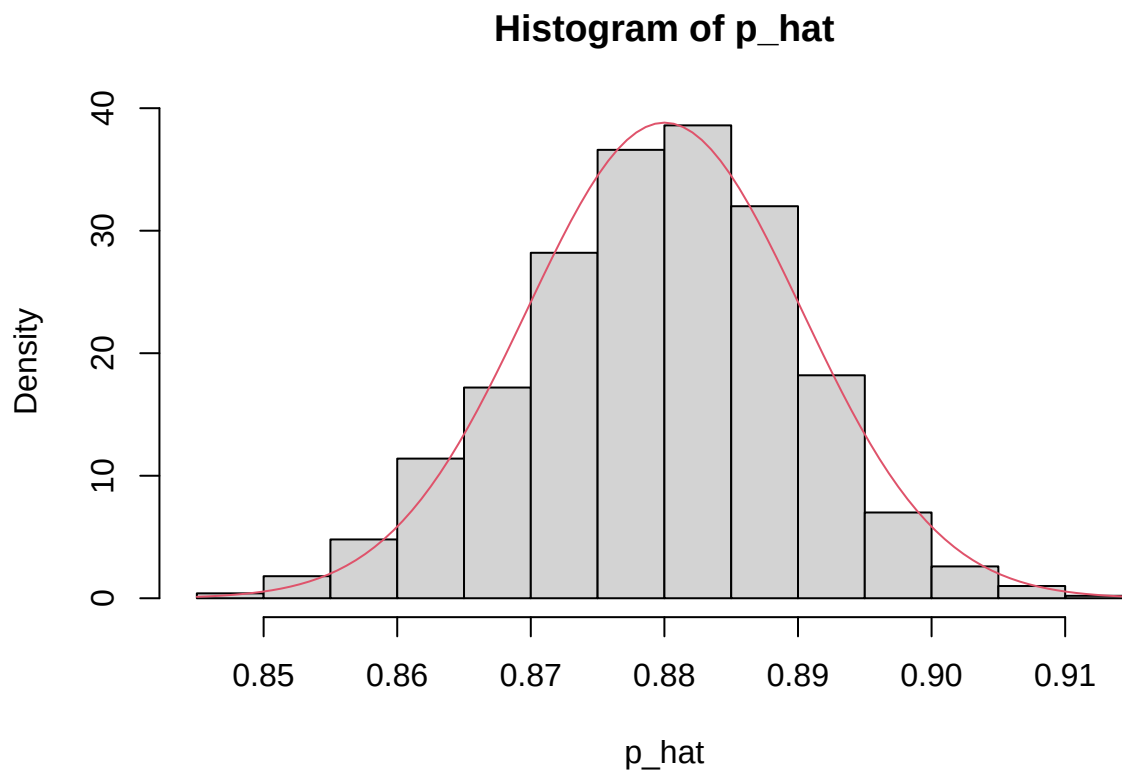
```
# Varianza della propo campionaria  
var(p_hat)
```

```
## [1] 0.0001062908
```

```
# Vera varianza  
p*(1-p)/n_camp
```

```
## [1] 0.0001056
```

```
# Istogramma della distribuzione della proporzione campionaria + curva che approssima la distribuzione  
hist(p_hat,prob=T)  
curve(dnorm(x,mean=p,sd=sqrt(p*(1-p)/n_camp)),add=T,col=2)
```



Binomiale

```

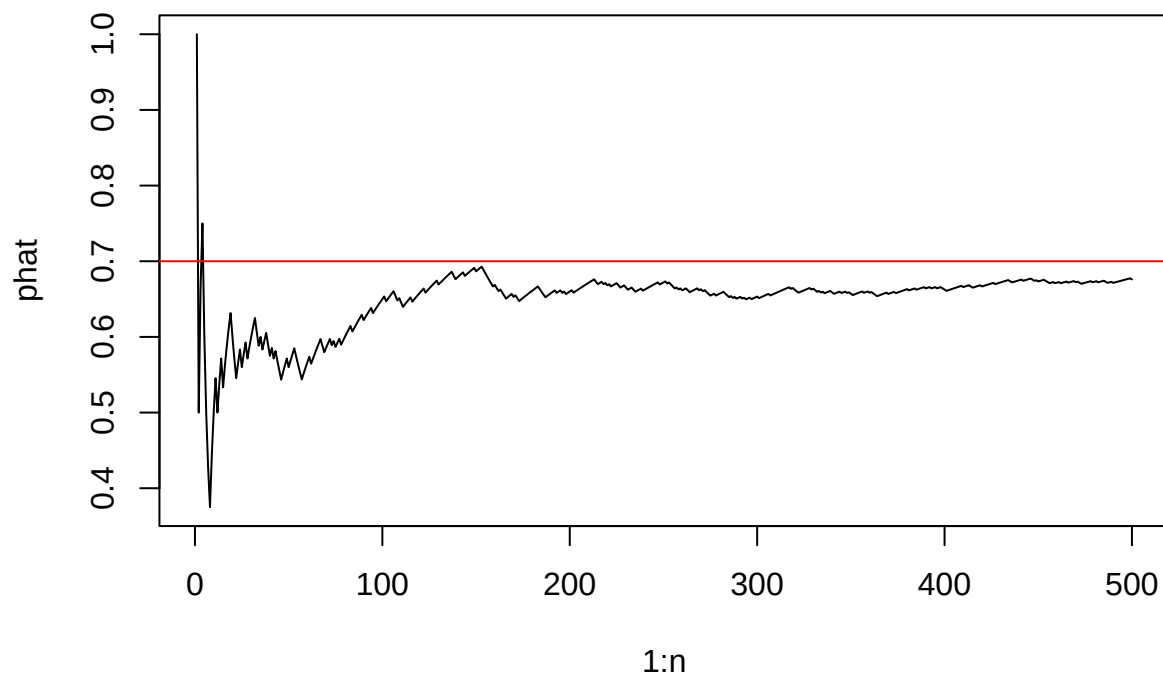
p = 0.7 # probabilità di ottenere testa
n = 500 # lanci moneta truccata

# Simulare 500 lanci della moneta truccata in cui la probabilità di ottenere testa è pari a 0.7.
set.seed(06)
campione = rbinom(n,prob=p,size=1)
# 1 successo è Testa
# 0 insuccesso è Croce

# Si calcoli la successione di valori della media campionaria (Calcolare la proporzione di casi in cui .
phat = cumsum(campione)/(1:n)

# Rappresentare su un grafico tali successioni, confrontandole con il valore atteso secondo la teoria.
plot(1:n,phat,type="l")
abline(h=p,col="red")

```



Uniforme

```

# Si studi l'andamento della legge della somma e della media campionaria di n variabili aleatorie indip
#Si simulino N=1000 realizzazioni della media e della somma di n=50,100,200,500 variabili aleatorie, Un

```

```
N = 1000 # numero di campioni
n = c(50,100,200,500)#numerosità campionarie
```

```
media = matrix(0,ncol = 4,nrow = N)
somma = matrix(0,ncol = 4,nrow = N)
```

```
for(j in 1:4){
  for(i in 1:N){
    dat = runif(n[j],0,1)
    media[i,j] = mean(dat)
    somma[i,j] = sum(dat)
  }
}
```

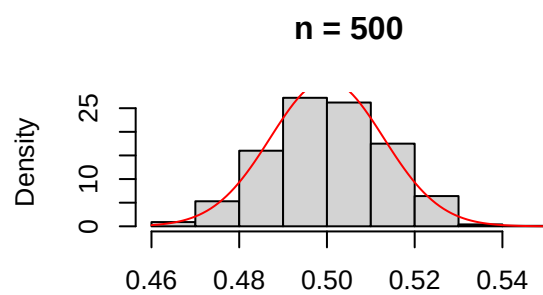
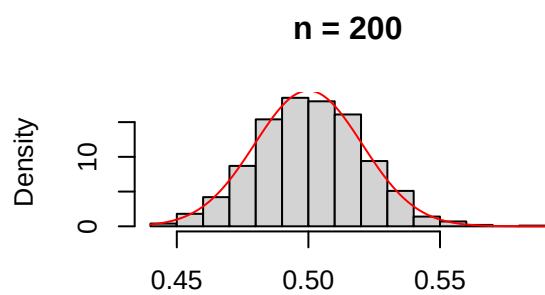
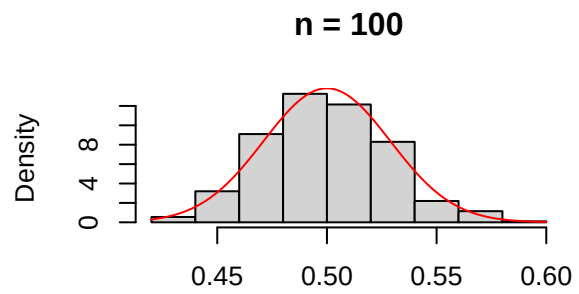
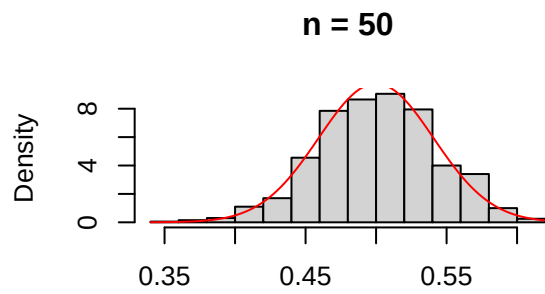
```
# Studiare la gaussianità della distribuzione delle medie con un istogramma e aggiungere la densità del
par(mfrow=c(2,2))
```

```
hist(media[,1], prob=T,main="n = 50",xlab="")
curve(dnorm(x,0.5,sqrt(1/(50*12))),col="red",add=T)
```

```
hist(media[,2], prob=T,main="n = 100",xlab="")
curve(dnorm(x,0.5,sqrt(1/(100*12))),col="red",add=T)
```

```
hist(media[,3], prob=T,main="n = 200",xlab="")
curve(dnorm(x,0.5,sqrt(1/(200*12))),col="red",add=T)
```

```
hist(media[,4], prob=T,main="n = 500",xlab="")
curve(dnorm(x,0.5,sqrt(1/(500*12))),col="red",add=T)
```



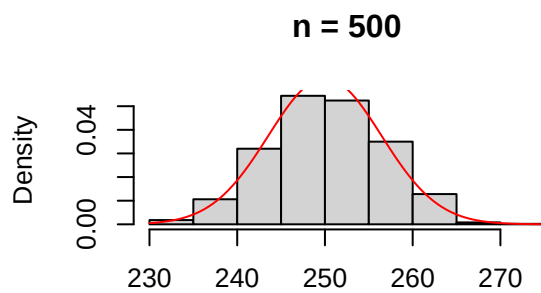
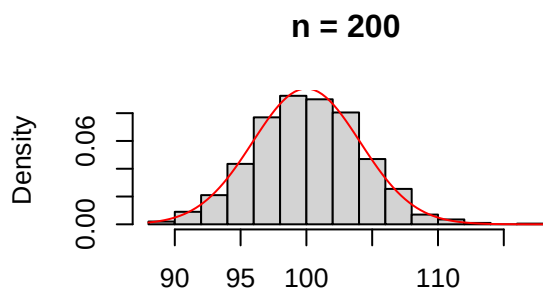
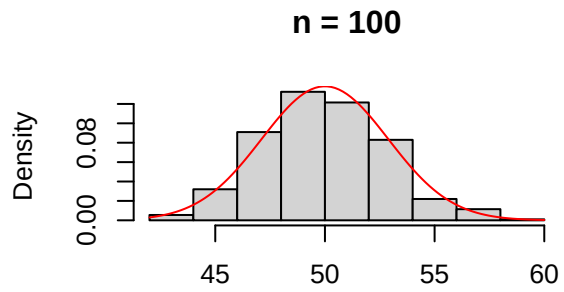
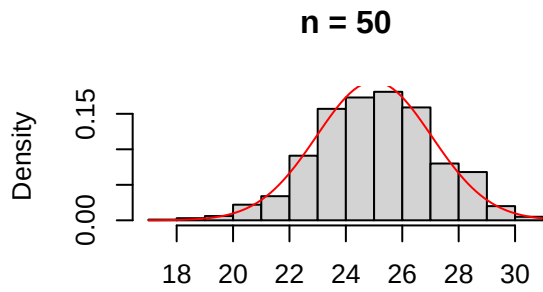
Studiare la gaussianità della distribuzione delle somme con un istogramma e aggiungere la densità del
`par(mfrow=c(2,2))`

```
hist(somma[,1], prob=T, main="n = 50", xlab="")
curve(dnorm(x, 50*0.5, sqrt(50/12)), col="red", add=T)

hist(somma[,2], prob=T, main="n = 100", xlab="")
curve(dnorm(x, 100*0.5, sqrt(100/12)), col="red", add=T)

hist(somma[,3], prob=T, main="n = 200", xlab="")
curve(dnorm(x, 200*0.5, sqrt(200/12)), col="red", add=T)

hist(somma[,4], prob=T, main="n = 500", xlab="")
curve(dnorm(x, 500*0.5, sqrt(500/12)), col="red", add=T)
```



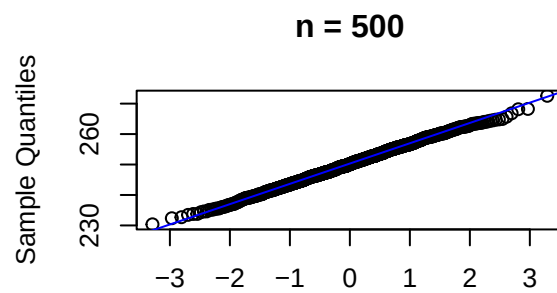
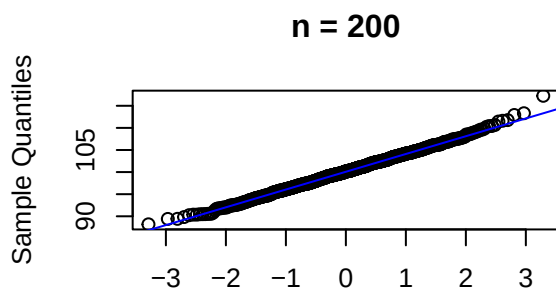
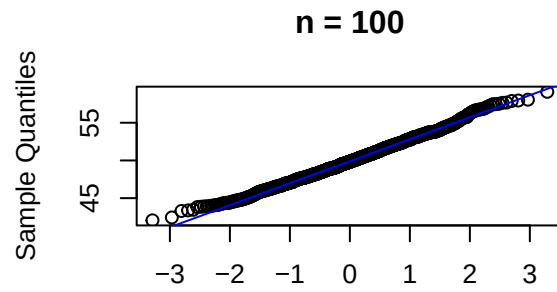
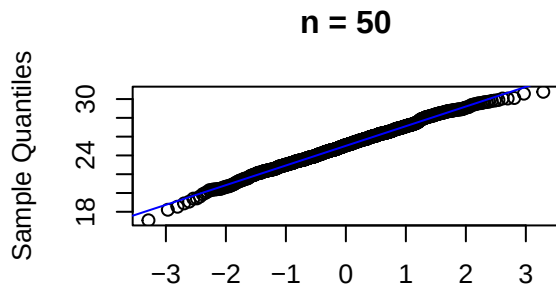
```
# Studiare la gaussianità della distribuzione delle somme con il QQ plot.
par(mfrow=c(2,2))
```

```
qqnorm(somma[,1], main="n = 50",xlab="")
qqline(somma[,1],col="blue")
```

```
qqnorm(somma[,2], main="n = 100",xlab="")
qqline(somma[,2],col="blue")
```

```
qqnorm(somma[,3], main="n = 200",xlab="")
qqline(somma[,3],col="blue")
```

```
qqnorm(somma[,4], main="n = 500",xlab="")
qqline(somma[,4],col="blue")
```



```
# oppure
library(car)
```

```
## Warning: package 'car' was built under R version 4.4.2
```

```
## Loading required package: carData
```

```
## Warning: package 'carData' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'car'
```

```
## The following objects are masked from 'package:mosaic':
```

```
##
```

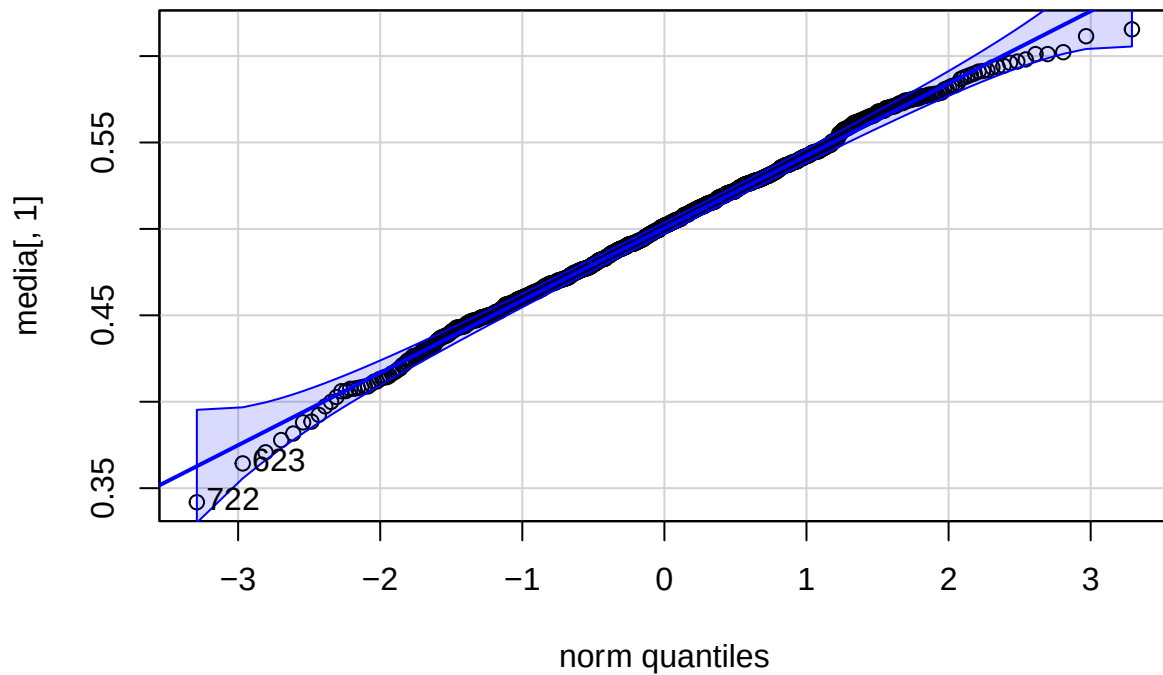
```
##   deltaMethod, logit
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##   recode
```

```
qqPlot(media[,1])
```



```
## [1] 722 623
```

INTERVALLI DI CONFIDENZA

```
# Si consideri il data-base REDDITI relativo a un campione di 12690 individui percettori di reddito in .
library(readxl)
```

```
## Warning: package 'readxl' was built under R version 4.4.2
```

```
REDDITI <- read_excel("REDDITI.xls")
attach(REDDITI)

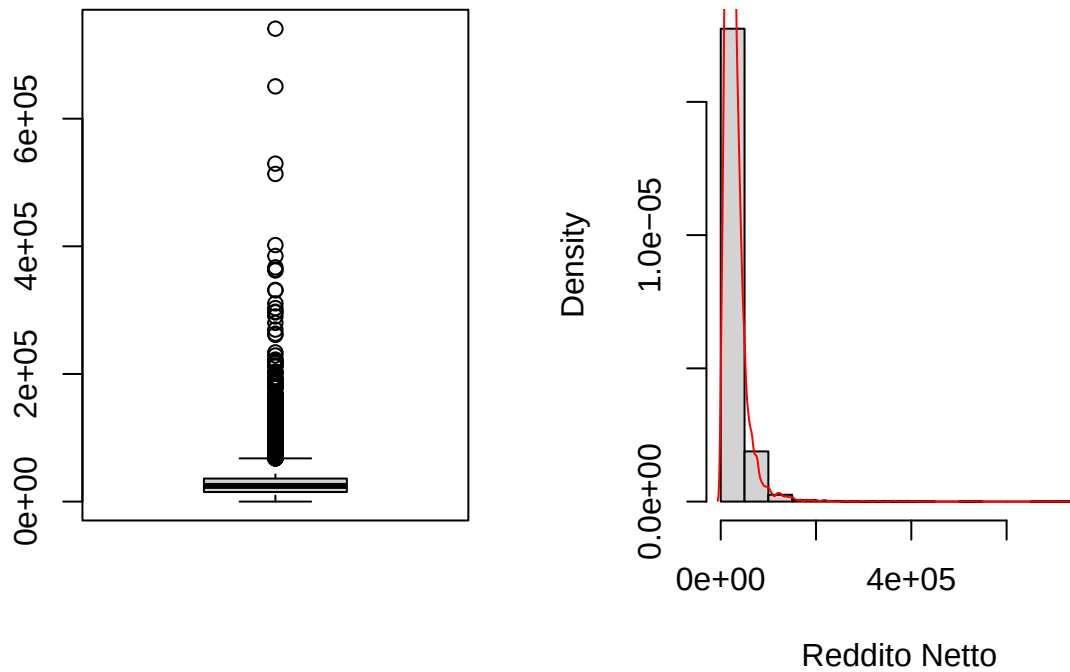
# Analizzare la variabile Reddito netto.
summary('Reddito Netto')
```

```
##      Length      Class      Mode
##          1 character character
```

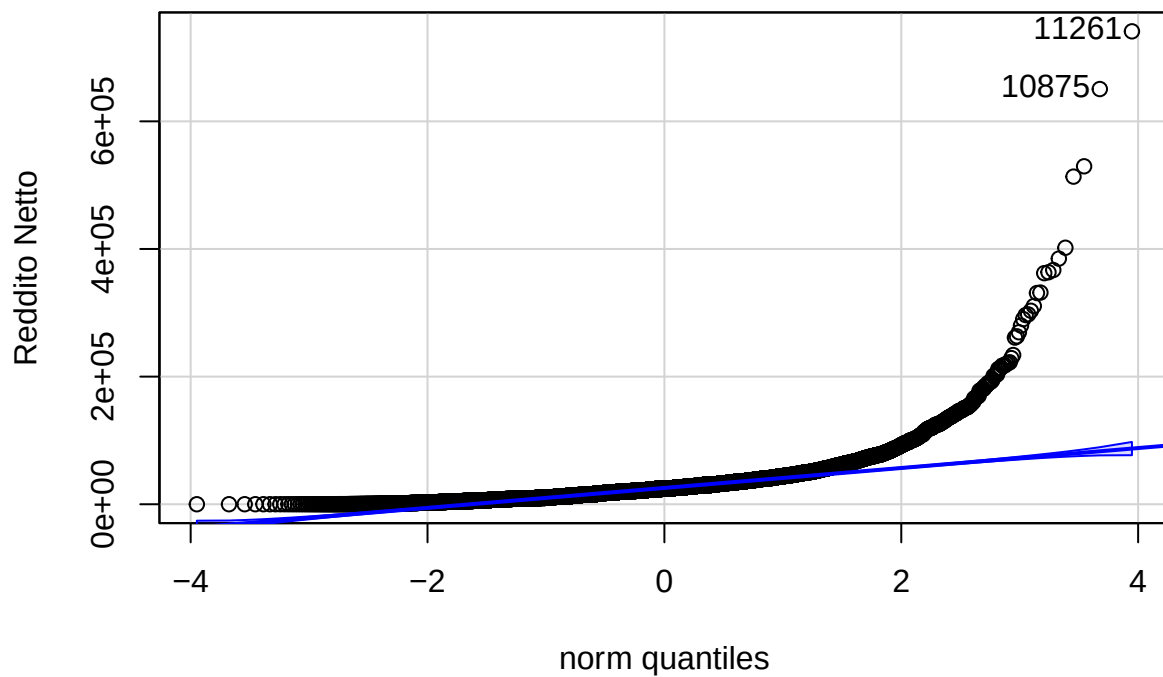
Rappresentazione grafica

```
par(mfrow=c(1,2))
boxplot(`Reddito Netto`)
hist(`Reddito Netto`,prob=T)
lines(density(`Reddito Netto`),col="red")
```

Histogram of Reddito Netto



```
library(car)
qqPlot(`Reddito Netto`)
```



```
## [1] 11261 10875
```

Variabili numeriche

```
# Summary
summary(`Reddito Netto`)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##         0  15000   24547   29453   36076   741027
```

```
# Skewness
library(moments)
skewness(`Reddito Netto`)
```

```
## [1] 6.615117
```

```
# Kurtosi
kurtosis(`Reddito Netto`)
```

```
## [1] 105.2432
```

Intervallo di confidenza

```
# Formula generica
intervallo_conf = function(x,alpha){
  n = length(x)
  xbar = mean(x)
  s = sd(x)
  t_alpha_mezzi = qt(1-alpha/2,n-1)
  # t_alpha_mezzi = qt(alpha/2,n-1,lower.tail=F)
  primo_estremo = xbar-t_alpha_mezzi*s/sqrt(n)
  secondo_estremo = xbar+t_alpha_mezzi*s/sqrt(n)
  return(list(primo=primo_estremo,secondo=secondo_estremo))
}

# Stimare l'intervallo di confidenza al 95% per il reddito medio.
intervallo_conf(`Reddito Netto`,0.05)
```

```
## $primo
## [1] 28984.33
##
## $secondo
## [1] 29922.51
```

```
# I.C. Varianza
intervallo_conf_varianza = function(x,alpha){
  n = length(x)
  s2_c = var(x)
  chi2_alpha_mezzi = qchisq(alpha/2,n-1,lower.tail = F)
  chi2_1meno_alpha_mezzi = qchisq(1-alpha/2,n-1,lower.tail = F)
  primo_estremo = (n-1)*s2_c/chi2_alpha_mezzi
  secondo_estremo = (n-1)*s2_c/chi2_1meno_alpha_mezzi
  return(list(primo=primo_estremo,secondo=secondo_estremo))
}
```

```
# Stimare l'intervallo di confidenza al 90% per la varianza del reddito.
intervallo_conf_varianza(`Reddito Netto`,alpha = 0.10)
```

```
## $primo
## [1] 711256954
##
## $secondo
## [1] 741264224
```

```
# I.C. Proporzioni
intervallo_conf_prop = function(x,alpha){
  phat = mean(x)
  n = length(x)
  z_alpha_mezzi = qnorm(alpha/2,lower.tail = F)
  primo_estremo = phat-z_alpha_mezzi*sqrt(phat*(1-phat)/n)
  secondo_estremo = phat+z_alpha_mezzi*sqrt(phat*(1-phat)/n)
  return(list(primo=primo_estremo,secondo=secondo_estremo))
}
```

```
# Stimare l'intervallo di confidenza al 95% per la proporzione di individui di sesso maschile.
intervallo_conf_prop(SEX,0.05)
```

```
## $primo
## [1] 0.5211689
##
## $secondo
## [1] 0.5385455
```

Stimare l'intervallo di confidenza per due classi

```
tapply(`Reddito Netto`, SEX, intervallo_conf,alpha=0.05)
```

```
## $`0`
## $`0`$primo
## [1] 20953.12
##
## $`0`$secondo
## [1] 21803.98
##
##
## $`1`
## $`1`$primo
## [1] 35857.34
##
## $`1`$secondo
## [1] 37379.19
```

```
# Summary
tapply(`Reddito Netto`, SEX, summary)
```

```
## $`0`
##      Min.  1st Qu.  Median    Mean 3rd Qu.    Max.
##      14.5  10010.0 19200.0 21378.5 27527.7 401925.0
##
## $`1`
##      Min.  1st Qu.  Median    Mean 3rd Qu.    Max.
##         0   20580   30082   36618   43287   741027
```

```
# Skewness
tapply(`Reddito Netto`, SEX, skewness)
```

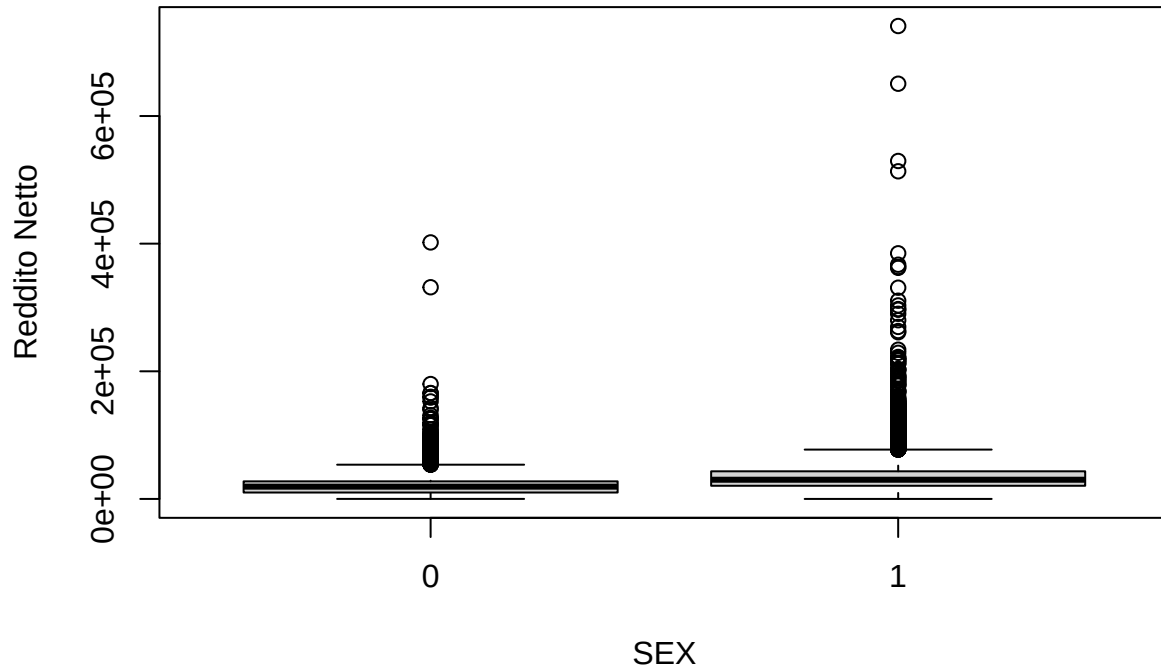
```
##           0           1
## 4.997379 6.453633
```

```
# Curtosi
tapply(`Reddito Netto`, SEX, kurtosis)
```

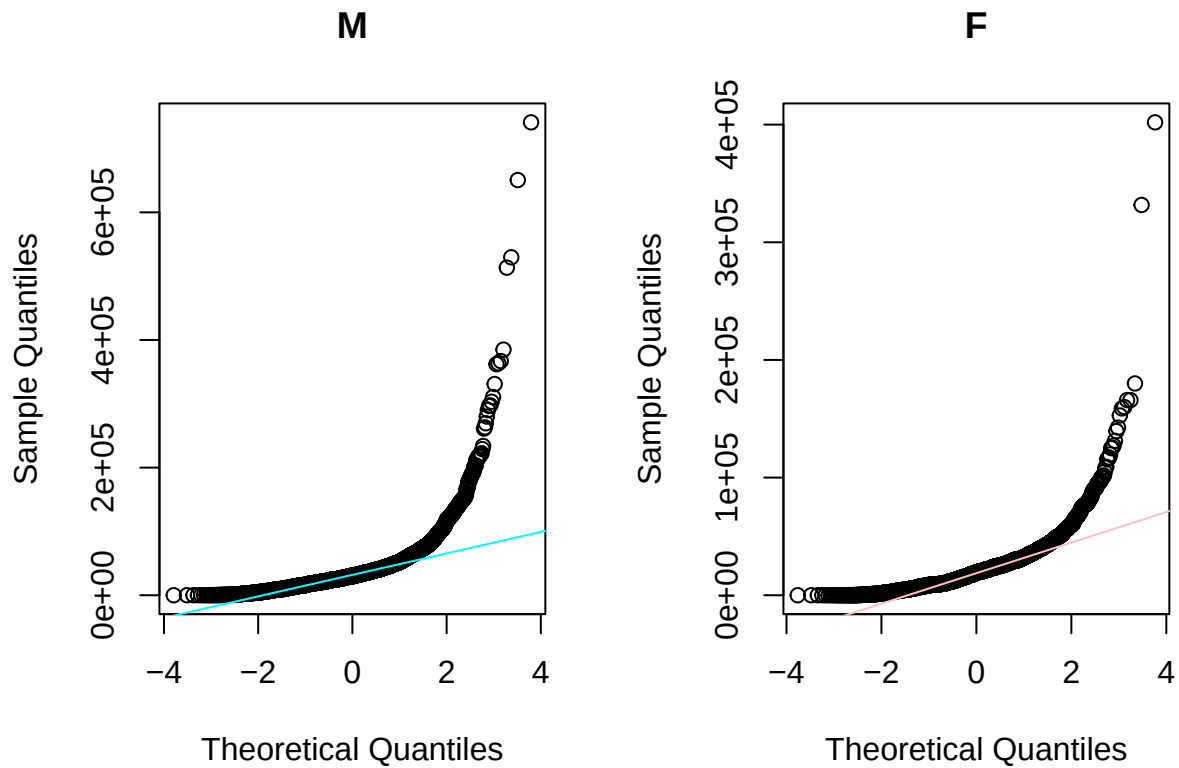
```
##           0           1
## 76.28010 91.46296
```

Visualizzazione

```
# Boxplot  
boxplot(`Reddito Netto`~SEX)
```



```
# Q-Q Plot  
par(mfrow=c(1,2))  
qqnorm(`Reddito Netto`[SEX==1],main="M")  
qqline(`Reddito Netto`[SEX==1],col="cyan")  
  
qqnorm(`Reddito Netto`[SEX==0],main="F")  
qqline(`Reddito Netto`[SEX==0],col="pink")
```



TEST - 1

Es. 1

```
#Si consideri il file "glucosio.txt". I dati consistono nella misura della concentrazione di glucosio n
glucosio <- read.csv("glucosio.txt", sep="")
attach(glucosio)
```

```
# Verificare la normalità dei dati.
summary(Glucosio)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      2.200   3.600   4.000   4.055   4.525   6.000
```

```
# Skewness
library(moments)
skewness(Glucosio)
```

```
## [1] 0.1205697
```

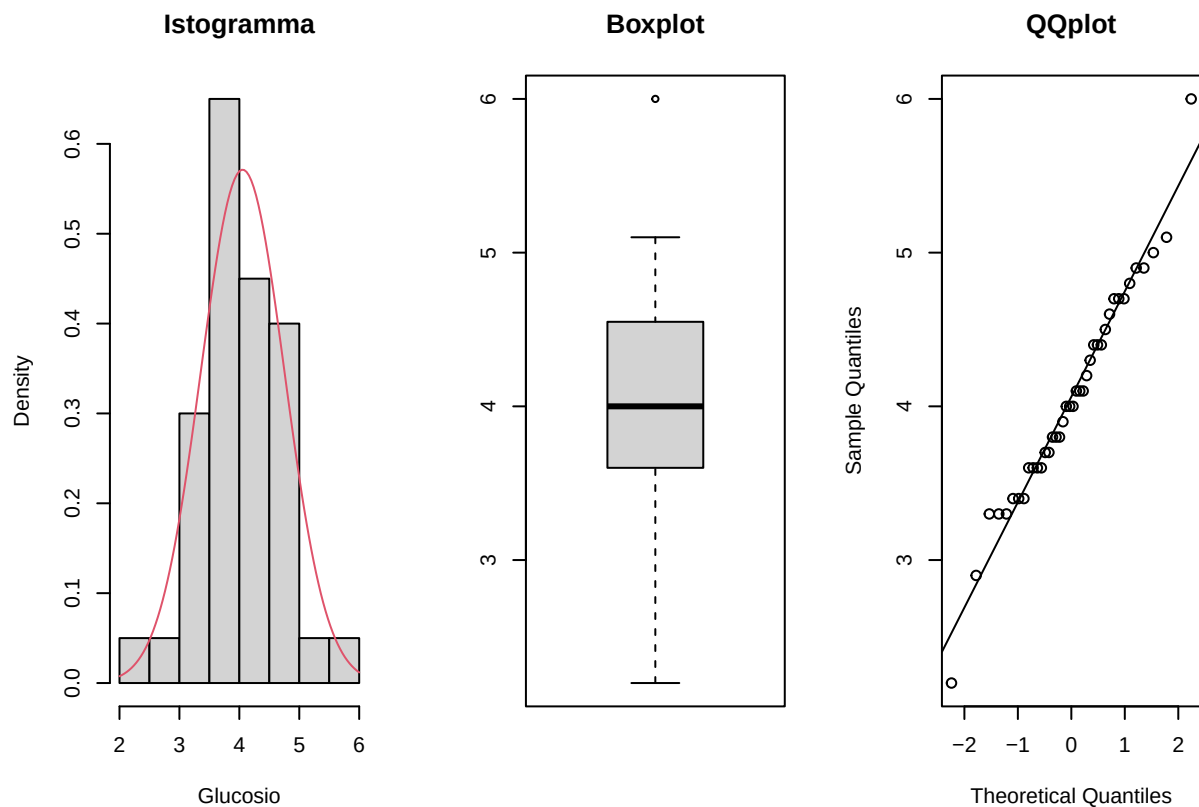
```
# Curtosi
kurtosis(Glucosio)
```

```
## [1] 3.725064
```

```
# Visualizzazione
par(mfrow=c(1,3))
hist(Glucosio,prob=T,main="Istogramma")
curve(dnorm(x,mean(Glucosio),sd(Glucosio)),add=T,col=2)

boxplot(Glucosio,main="Boxplot")

qqnorm(Glucosio,main="QQplot")
qqline(Glucosio)
```



I.C. Media

```
# Si costruisca un intervallo di confidenza di livello 99% per la vera media della concentrazione di gl
intervallo_media = function(x,alpha){
  n = length(x)
  t_alpha_mezzi= qt(alpha/2,n-1,lower.tail = F)
  primo = mean(x)-t_alpha_mezzi*sd(x)/sqrt(n)
```

```

    secondo = mean(x)+t_alpha_mezzi*sd(x)/sqrt(n)
    return(list(primo_estremo=primo,secondo_estremo=secondo))
}

```

```

intervallo_media(Glucosio,alpha=0.01)

```

```

## $primo_estremo
## [1] 3.756003
##
## $secondo_estremo
## [1] 4.353997

```

```

# Si esegua il seguente test d'ipotesi  $H_0:\mu=4$  contro  $H_1:\mu\neq 4$  per decidere se la vera concentrazione m
mu0 = 4
alpha = 0.05
n = length(Glucosio)

```

```

# Formula matematica
(t_alpha_mezzi= qt(alpha/2,n-1,lower.tail = F))

```

```

## [1] 2.022691

```

```

(t_oss = (mean(Glucosio)-mu0)/(sd(Glucosio)/sqrt(n)))

```

```

## [1] 0.4981168

```

```

(p_value=2*min(pt(t_oss,n-1),1-pt(t_oss,n-1)))

```

```

## [1] 0.6211992

```

Test

```

# Con funzione test
t.test(Glucosio,mu=mu0,alternative = "two.sided",conf.level = 0.95)

```

```

##
## One Sample t-test
##
## data: Glucosio
## t = 0.49812, df = 39, p-value = 0.6212
## alternative hypothesis: true mean is not equal to 4
## 95 percent confidence interval:
## 3.831663 4.278337
## sample estimates:
## mean of x
## 4.055

```

```
# Calcolo della statistica test per verificare l'ipotesi: H0:varianza=1 contro H1:varianza!=1. Calcolando
n = length(Glucosio)
sigma2_0 = 1# valore di sigma^2 sotto l'ipotesi nulla
(k_oss = (n-1)*var(Glucosio)/sigma2_0)
```

```
## [1] 19.019
```

```
# Matematicamente
alpha = 0.05
(chiquadro_1meno_alpha_mezzi = qchisq(1-alpha/2,n-1,lower.tail = F))
```

```
## [1] 23.65432
```

```
(chiquadro_alpha_mezzi = qchisq(alpha/2,n-1,lower.tail = F))
```

```
## [1] 58.12006
```

```
k_oss < chiquadro_1meno_alpha_mezzi | k_oss>chiquadro_alpha_mezzi
```

```
## [1] TRUE
```

```
(p_value = 2*min(pchisq(k_oss,n-1),1-pchisq(k_oss,n-1)))
```

```
## [1] 0.005895498
```

VarTest

```
# Con funzione di r varTest
library(EnvStats)
```

```
## Warning: package 'EnvStats' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'EnvStats'
```

```
## The following object is masked from 'package:car':
```

```
##
```

```
## qqPlot
```

```
## The following object is masked from 'package:mosaic':
```

```
##
```

```
## iqr
```

```
## The following objects are masked from 'package:moments':
```

```
##
```

```
## kurtosis, skewness
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## predict, predict.lm
```

```
varTest(Glucosio,alternative = "two.sided",conf.level = 0.95,sigma.squared = sigma2_0)
```

```
##
## Results of Hypothesis Test
## -----
##
## Null Hypothesis:                variance = 1
##
## Alternative Hypothesis:         True variance is not equal to 1
##
## Test Name:                     Chi-Squared Test on Variance
##
## Estimated Parameter(s):        variance = 0.4876667
##
## Data:                          Glucosio
##
## Test Statistic:                Chi-Squared = 19.019
##
## Test Statistic Parameter:      df = 39
##
## P-value:                       0.005895498
##
## 95% Confidence Interval:       LCL = 0.3272364
##                               UCL = 0.8040390
```

Es. 2

```
# Luca invita a casa 30 amici per festeggiare il suo diciottesimo compleanno. La variabile "età", riportata nella tabella sottostante, rappresenta l'età degli invitati.
età = c(13, 15, 19, 20, 22, 16, 17, 14, 18, 18, 19, 20, 23, 24, 21, 25, 23, 14,
        13, 17, 18, 19, 15, 19, 20, 21, 22, 23, 27, 25)
età
```

```
## [1] 13 15 19 20 22 16 17 14 18 18 19 20 23 24 21 25 23 14 13 17 18 19 15 19 20
## [26] 21 22 23 27 25
```

```
# Stima della proporzione di maggiorenni
maggiorenni = età[età>=18]

(phat = length(maggiorenni)/length(età))
```

```
## [1] 0.7
```

```
# Calcolo dell'intervallo di confidenza a livello 95% per la proporzione di maggiorenni p.
alpha = 0.05
p0 = 0.6
n = length(età)
zalpha_mezzi=qnorm(1-alpha/2)

# 1° Estremo
(primo_estremo = phat-zalpha_mezzi*sqrt((phat*(1-phat)/n)))
```

```
## [1] 0.5360176
```

```
# 2° Estremo  
(secondo_estremo = phat+zalpha_mezzi*sqrt((phat*(1-phat)/n)))
```

```
## [1] 0.8639824
```

```
# Calcolo della statistica test per verificare l'ipotesi: H0:p=0.6 contro H1:p>0.6.  
  
# Formule matematiche  
(z_oss= (phat-p0)/sqrt(p0*(1-p0)/n))
```

```
## [1] 1.118034
```

```
# Verifica regione rifiuto  
zalpha = qnorm(1-alpha)  
z_oss>zalpha
```

```
## [1] FALSE
```

```
# P-Value del test  
(p_value=1-pnorm(z_oss))
```

```
## [1] 0.1317762
```

Prop.Test

```
# funzione prop.test  
successi = length(maggiorenni)  
  
prop.test(successi,n,alternative = "greater",conf.level = 1-alpha,p=p0,correct = F)
```

```
##  
## 1-sample proportions test without continuity correction  
##  
## data: successi out of n  
## X-squared = 1.25, df = 1, p-value = 0.1318  
## alternative hypothesis: true p is greater than 0.6  
## 95 percent confidence interval:  
## 0.5506175 1.0000000  
## sample estimates:  
## p  
## 0.7
```

TEST - 2

Esercizio 1

Verifica normalità

```
## i due campioni distinti per sesso
temp.m <- Temperatura[which(Sesso=='U')]
temp.f <- Temperatura[which(Sesso=='D')]
(n <- length(temp.m))
```

```
## [1] 65
```

```
length(temp.f)
```

```
## [1] 65
```

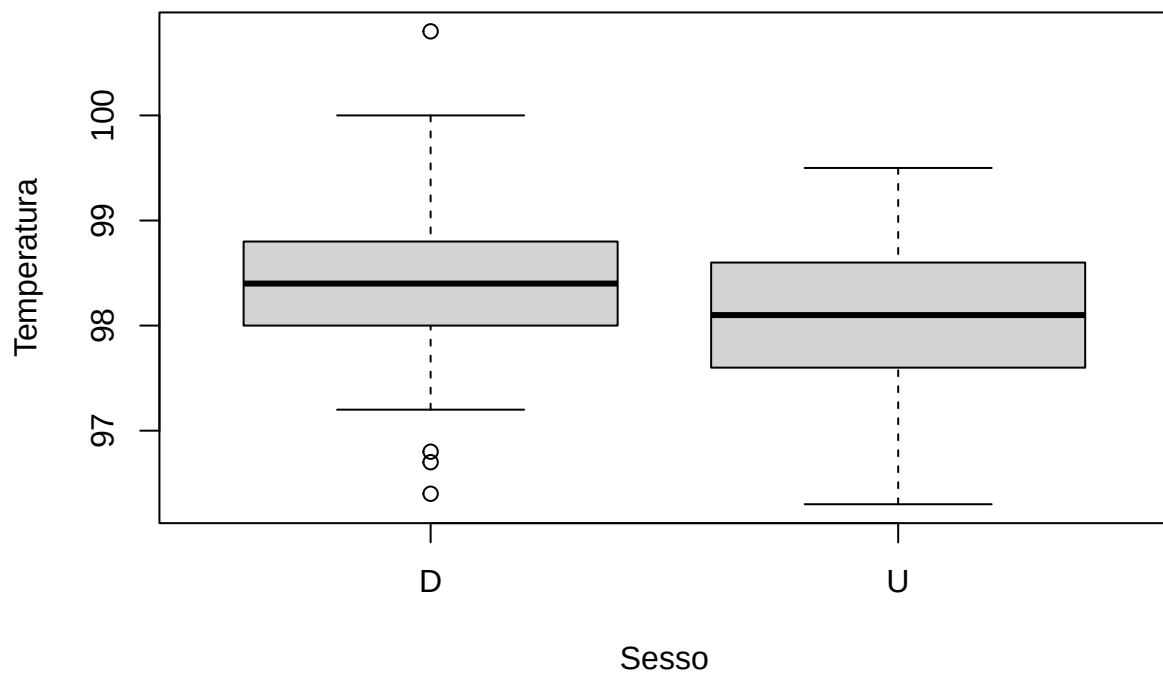
```
# media uomini
(media.m <- mean(temp.m))
```

```
## [1] 98.10462
```

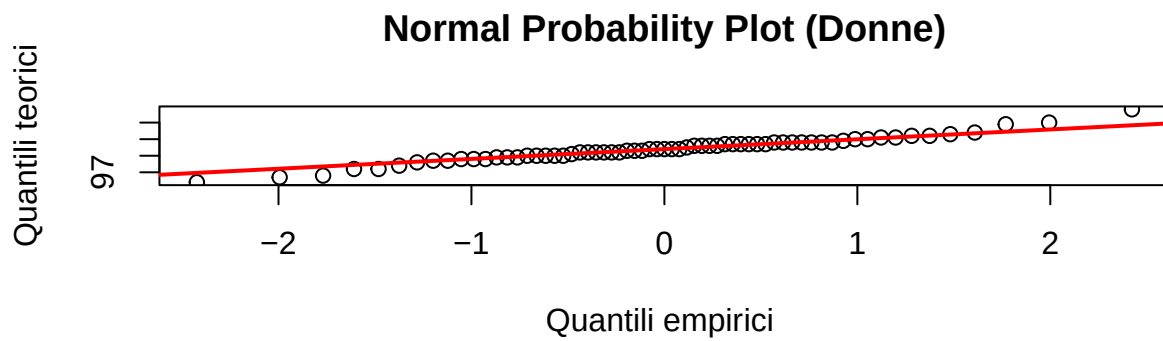
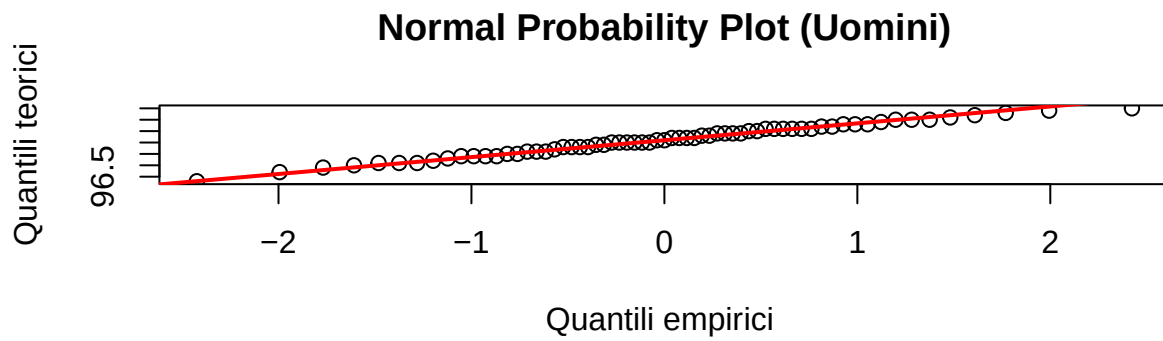
```
# media donne
(media.f <- mean(temp.f))
```

```
## [1] 98.39385
```

```
boxplot(Temperatura~Sesso)
```



```
par(mfrow=c(2,1))
qqnorm(temp.m, main='Normal Probability Plot (Uomini)', xlab="Quantili empirici ",ylab="Quantili teorici")
qqline(temp.m,col="red",lwd=2)
qqnorm(temp.f, main='Normal Probability Plot (Donne)', xlab="Quantili empirici ",ylab="Quantili teorici")
qqline(temp.f,col="red",lwd=2)
```



Test sulle varianze

```
# varianza uomini
var(temp.m)
```

Matematicamente

```
## [1] 0.4882596
```

```
# varianza donne
var(temp.f)
```

```
## [1] 0.552774
```

```
#parametri del test
alpha <- 0.05
(f.alpha1 <- qf(alpha/2,n-1,n-1))
```

```
## [1] 0.6099476
```

```

(f.alpha2 <- qf(1-alpha/2,n-1,n-1))

## [1] 1.639485

#calcolo della statistica test
(F_oss <- var(temp.m)/var(temp.f))

## [1] 0.8832897

#esito del test
F_oss < f.alpha1 | F_oss > f.alpha2

## [1] FALSE

#calcolo p-value
pvalue <- 2*min(pf(F_oss,n-1,n-1), 1-pf(F_oss,n-1,n-1))
pvalue

## [1] 0.6210837

var.test(temp.m, temp.f,alternative = 'two.sided', conf.level = 1-alpha)

```

Con funzione test

```

##
## F test to compare two variances
##
## data: temp.m and temp.f
## F = 0.88329, num df = 64, denom df = 64, p-value = 0.6211
## alternative hypothesis: true ratio of variances is not equal to 1
## 95 percent confidence interval:
## 0.5387604 1.4481404
## sample estimates:
## ratio of variances
## 0.8832897

```

Test sulle medie

```

# differenza campionaria
diff.camp <- media.f - media.m

# varianza comune
var.pooled <- ((n - 1)*var(temp.m) + (n - 1)*var(temp.f))/(2*n - 2)
alpha <- 0.05
t.alpha.2 <- qt(1-alpha/2, 2*n-2)

# calcolo della statistica test:
(T_oss <- diff.camp/sqrt(2*var.pooled/n))

```

Matematicamente

```
## [1] 2.285435
```

```
#esito del test  
T_oss <- t.alpha.2 | T_oss > t.alpha.2
```

```
## [1] TRUE
```

```
#calcolo del p-value  
(pvalue <- 2*min(pt(T_oss,2*n-2),(1 - pt(T_oss,2*n-2))))
```

```
## [1] 0.02393188
```

```
t.test(temp.f, temp.m, mu = 0, var.equal = TRUE, conf.level = 1-alpha)
```

Con funzione test

```
##  
## Two Sample t-test  
##  
## data: temp.f and temp.m  
## t = 2.2854, df = 128, p-value = 0.02393  
## alternative hypothesis: true difference in means is not equal to 0  
## 95 percent confidence interval:  
## 0.03882216 0.53963938  
## sample estimates:  
## mean of x mean of y  
## 98.39385 98.10462
```

Esercizio 2

```
# media pos  
mean(Pos)
```

```
## [1] 89.08333
```

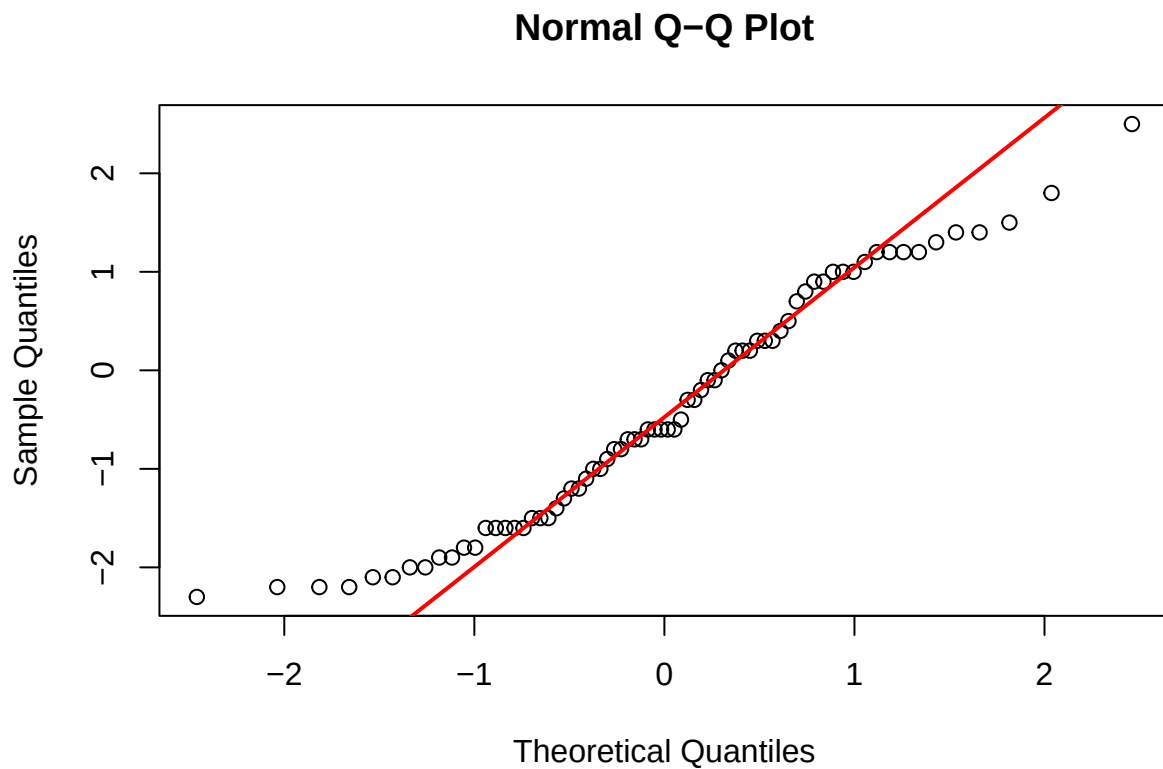
```
# media osm  
mean(Osm)
```

```
## [1] 89.49583
```

```
# differenza  
diff <- Pos - Osm
```

Verifica normalità

```
# si deduce che al centro è approssimabile alla normalità, ma a causa delle code che si distaccano non
qqnorm(diff)
qqline(diff,col="red",lwd=2)
```



Verificare esistenza di differenza statistica.

```
alpha = 0.01
(t.alpha.2 = qt(1-alpha/2,n-1))
```

Matematicamente

```
## [1] 2.646863
```

```
media.diff = mean(diff)
dev.std.diff = sd(diff)

# calcolo statistica test
(T_oss = media.diff/(dev.std.diff/sqrt(n)))
```

```
## [1] -2.890657
```

```
# esito del test
T_oss > t.alpha.2 | T_oss < t.alpha.2
```

```
## [1] TRUE
```

```
# calcolo del p-value
(pvalue = 2*min(pt(T_oss,n-1),1-pt(T_oss,n-1)))
```

```
## [1] 0.005096854
```

```
# paired indica la dipendenza tra le variabili
t.test(Pos, Osm, paired = TRUE, conf.level = 1-alpha)
```

Con funzione test

```
##
## Paired t-test
##
## data: Pos and Osm
## t = -2.8907, df = 71, p-value = 0.005097
## alternative hypothesis: true mean difference is not equal to 0
## 99 percent confidence interval:
## -0.79021034 -0.03478966
## sample estimates:
## mean difference
## -0.4125
```

Esercizio 3

[!] Tabella contingenza

```
table(istruzione)
```

```
##          Istruzione
## stato_civile analf diploma laurea obbligo
##   celibe      1      3      1      1
##   coniugato    0      4      2      1
##   separato     0      1      0      2
##   vedovo       1      0      1      2
```

Valutazione indipendenza con chi-quadro

```
chisq.test(table(istruzione))
```

```
## Warning in chisq.test(table(istruzione)): Chi-squared approximation may be
## incorrect
```

```
##
## Pearson's Chi-squared test
##
## data:  table(istruzione)
## X-squared = 8.2143, df = 9, p-value = 0.5127
```

```
# oppure
chisq.test(stato_civile, Istruzione)
```

```
## Warning in chisq.test(stato_civile, Istruzione): Chi-squared approximation may
## be incorrect
```

```
##
## Pearson's Chi-squared test
##
## data:  stato_civile and Istruzione
## X-squared = 8.2143, df = 9, p-value = 0.5127
```

```
# con test simulati
chisq.test(stato_civile, Istruzione, simulate.p.value = T)
```

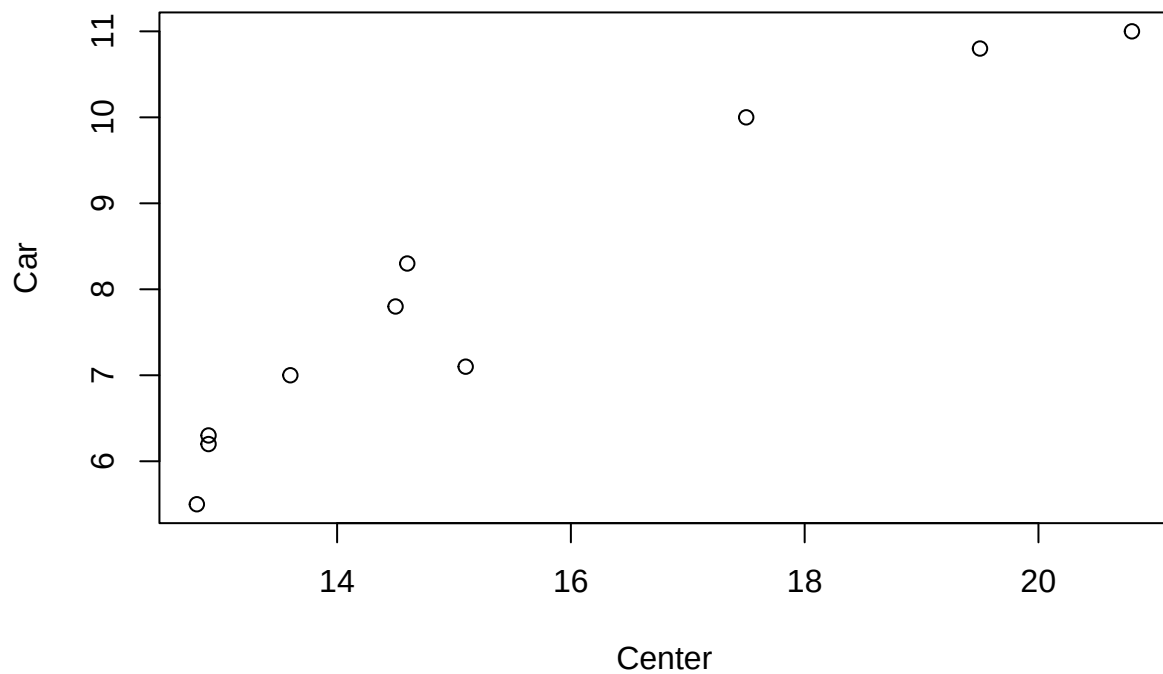
```
##
## Pearson's Chi-squared test with simulated p-value (based on 2000
## replicates)
##
## data:  stato_civile and Istruzione
## X-squared = 8.2143, df = NA, p-value = 0.5792
```

REGRESSIONE

Esercizio 1

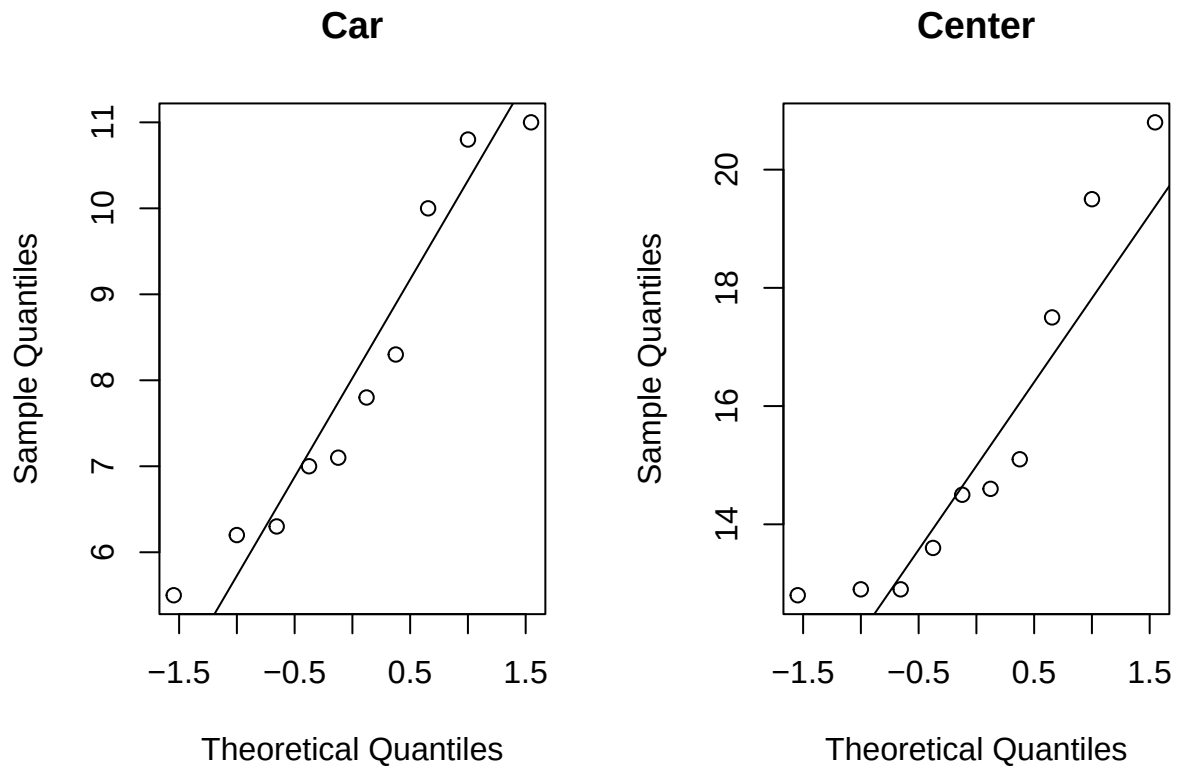
Rappresentazione grafica relazione

```
plot(Center, Car)
```



Terstare normalità variabili

```
# Rappresentazione grafica  
par(mfrow=c(1,2))  
qqnorm(Car,main="Car")  
qqline(Car)  
qqnorm(Center,main="Center")  
qqline(Center)
```



Controllo significatività relazione

```
cor.test(Car,Center,method = "spearman")
```

```
## Warning in cor.test.default(x, y, ...): Cannot compute exact p-value with ties
```

```
##
## Spearman's rank correlation rho
##
## data: x and y
## S = 6.519, p-value = 1.016e-05
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## 0.9604908
```

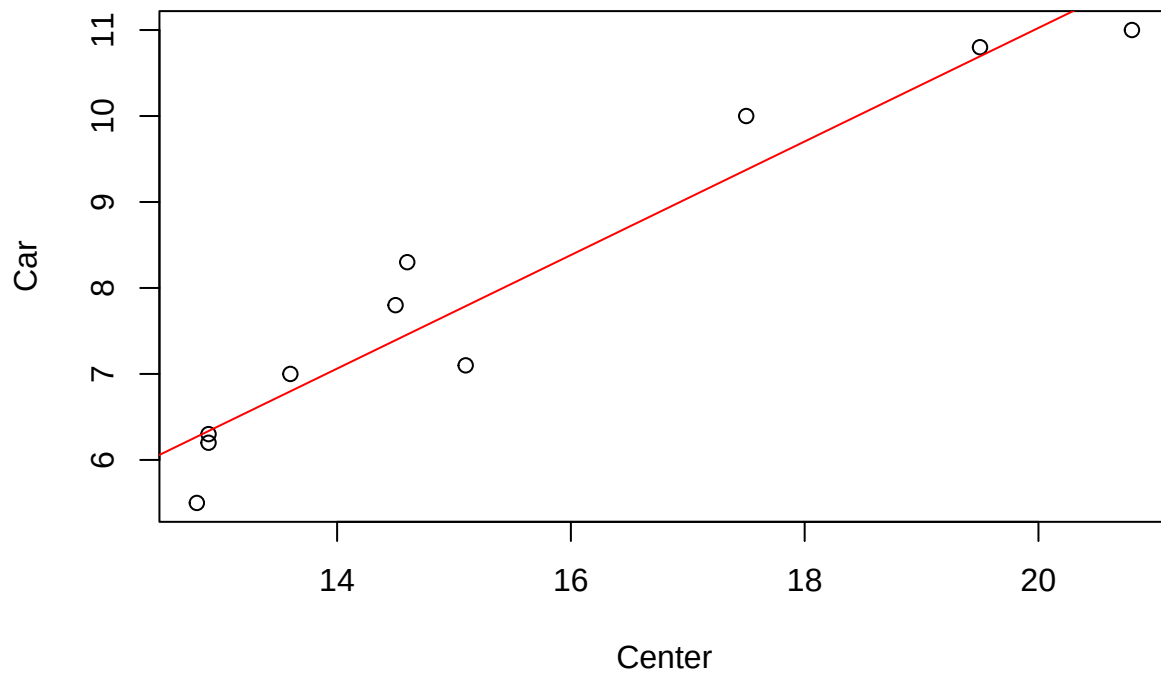
Stima modello regressione lineare

```
mod = lm(Car ~ Center,data=ciclisti)
summary(mod)
```

```
##
## Call:
## lm(formula = Car ~ Center, data = ciclisti)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.76990 -0.44846  0.03493  0.35609  0.84148
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.18247    1.05669  -2.065   0.0727 .
## Center       0.66034    0.06748   9.786 9.97e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5821 on 8 degrees of freedom
## Multiple R-squared:  0.9229, Adjusted R-squared:  0.9133
## F-statistic: 95.76 on 1 and 8 DF,  p-value: 9.975e-06
```

```
# Rappresentazione grafica retta di regressione
```

```
plot(Center,Car)
abline(mod,col="red")
```



Test significatività variabile Center

```
(p_value = summary(mod)$coefficients[2,4])
```

```
## [1] 9.974851e-06
```

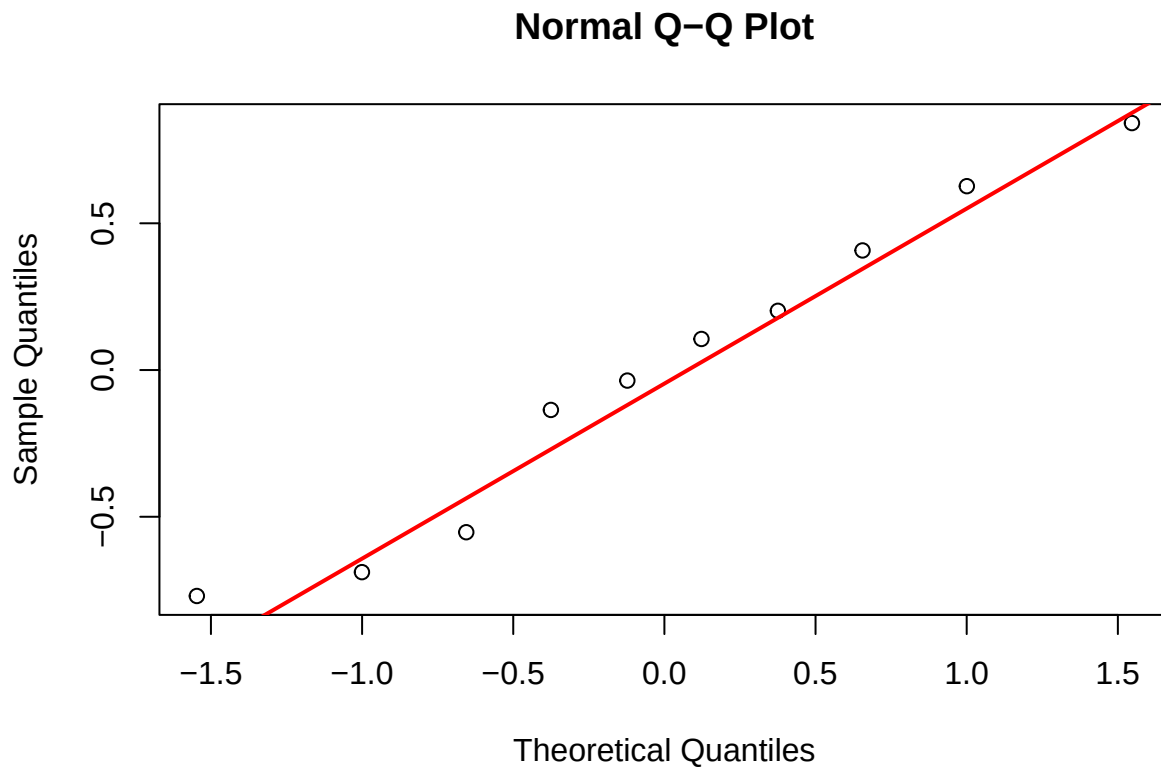
Il modello si adatta bene ai dati?

```
(R2 = summary(mod)$r.squared)
```

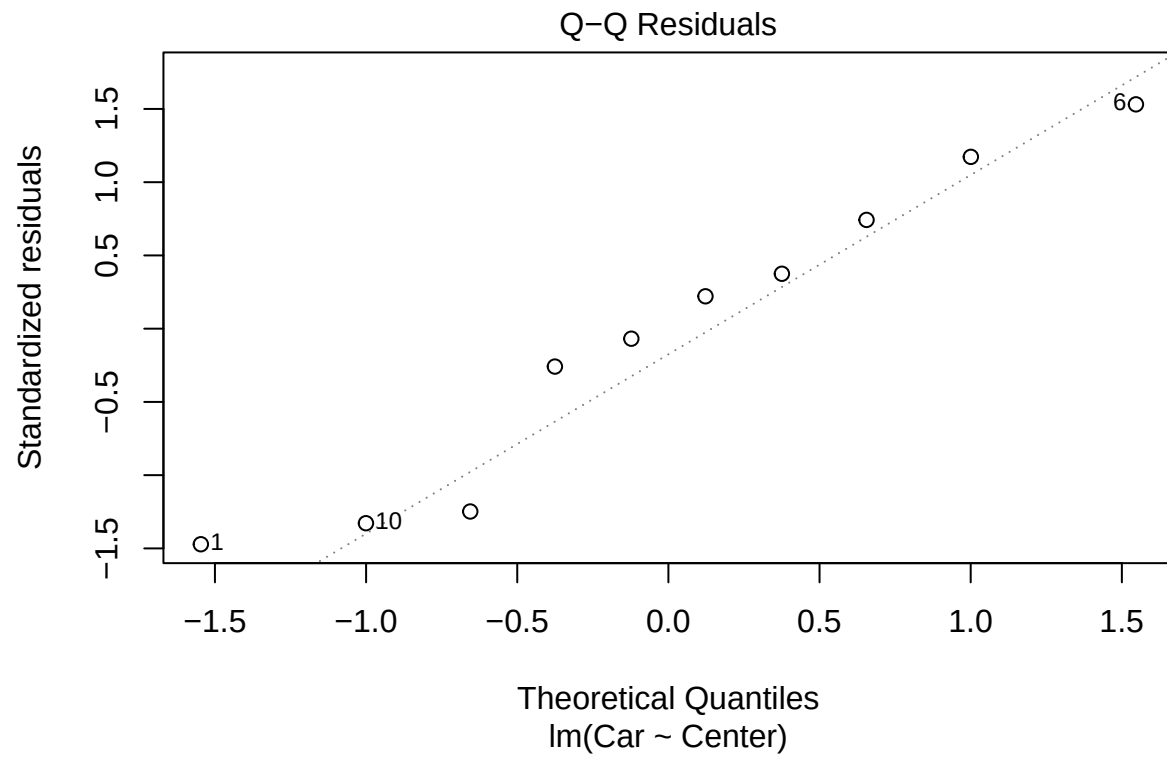
```
## [1] 0.922901
```

E' possibile ipotizzare una distribuzione normale per i residui del modello?

```
qqnorm(residuals(mod))  
qqline(residuals(mod), col='red', lwd=2)
```

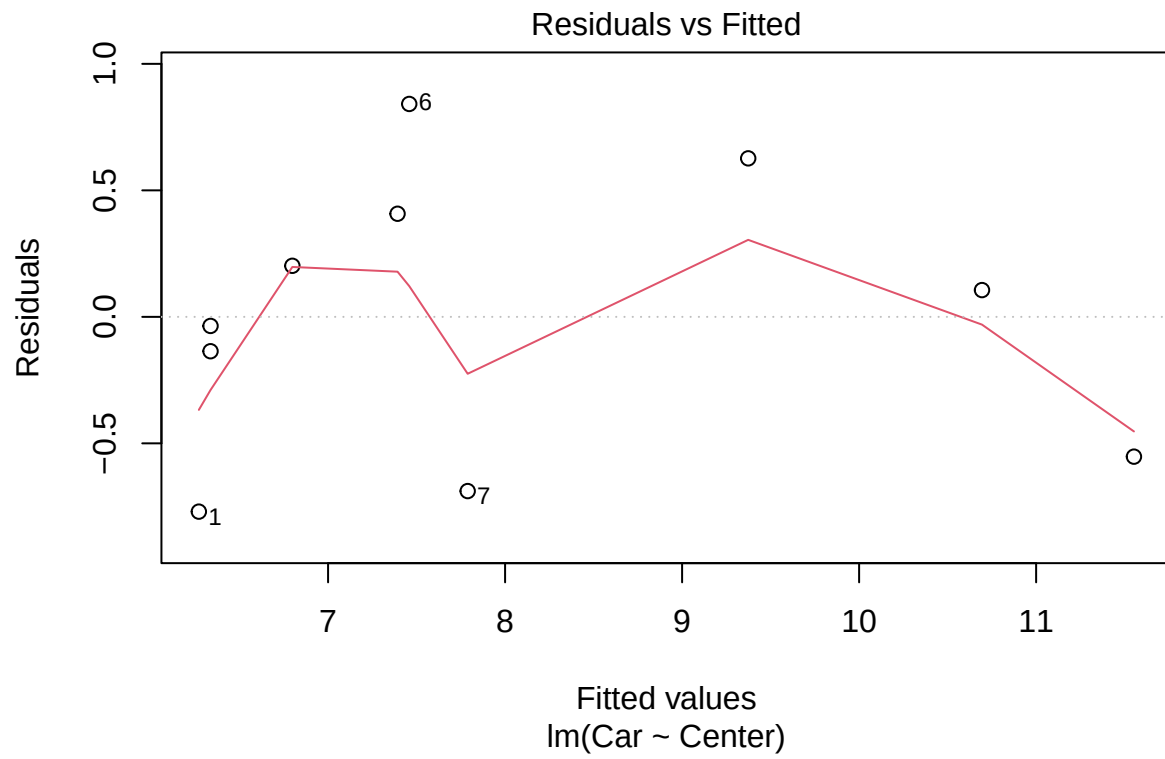


```
# oppure  
plot(mod,2)
```



Verifica eteroschedasticità

```
plot(mod,1)
```



Verifica outliers

```
plot(mod,4)
```

